CẨM NANG GITHUB

Từ cơ bản đến thành thạo • Hướng dẫn thực hành dễ hiểu

Ví dụ xuyên suốt: nix6242/DQY

Biên soạn và đối chiếu nguồn: 11/10/2026

Học từ đầu • Hiểu lý do • Tự làm và kiểm tra kết quả 18 chương / Bài tập DQY / Lộ trình 21 ngày

Mục lục

Bắt đầu từ đây....3
Chương 1. Hiểu GitHub bằng một dự án thật....5
Chương 2. Chuẩn bị tài khoản và quyền truy cập....7
Chương 3. Nhận biết giao diện và tạo kho khi cần....9
Chương 4. Viết README, quản lý tài liệu Word và lấy tệp về....11
Chương 5. Thực hành DQY: từ nhánh mới đến PR đã hợp nhất....13
Chương 6. Chọn clone, fork hay Download ZIP....16
Chương 7. Dùng GitHub Desktop: làm việc trên máy bằng giao diện....17
Chương 8. Dùng Git bằng dòng lệnh....20
Chương 9. Làm việc nhóm và quy trình mỗi ngày....24
Chương 10. Xử lý xung đột mà không mất công việc....27
Chương 11. Hoàn tác và cất công việc dở một cách an toàn....29
Chương 12. Sắp xếp kho, quản lý tệp Word và dùng .gitignore....31
Chương 13. Quản lý công việc bằng Issues, Projects và milestone....34
Chương 14. Actions, Pages và Releases: hiểu trước khi tự động hóa....36
Chương 15. Máy cá nhân, môi trường đám mây và tệp Word....39
Chương 16. Xử lý lỗi theo dấu hiệu, không đoán mò....41
Chương 17. Lộ trình 21 ngày và bài tổng hợp DQY....43
Chương 18. Tra nhanh: lệnh, thuật ngữ và câu hỏi thực tế....45
Nguồn tham khảo và cách tra cứu tiếp....47

Bắt đầu từ đây

Tài liệu này dành cho người bắt đầu hoặc đã quên cách dùng GitHub. Bạn không cần biết lập trình để học cách lưu tệp, quản lý phiên bản và làm việc cùng người khác. Các chương dùng tên nút tiếng Anh vì giao diện thường hiển thị như vậy; ngay sau đó có giải thích tiếng Việt.

Cách mở và tìm phần cần đọc trong Word

Mở tệp .docx bằng Microsoft Word hoặc ứng dụng tương thích. Trong Word trên máy, chọn **View** → **Navigation Pane** để thấy các đề mục ở bên cạnh; bấm đề mục để chuyển tới phần đó. Mục lục ở đầu tài liệu cũng có liên kết: bấm hoặc Ctrl + bấm theo ứng dụng đang dùng. Muốn tìm một từ như “push”, dùng Ctrl + F trên Windows hoặc Command + F trên macOS.

Nếu bạn sửa nội dung làm số trang thay đổi, trong Word chọn mục lục và dùng **Update Table / Update Field**; chọn cập nhật toàn bộ nếu có lựa chọn. Bố cục và số trang có thể thay đổi nhẹ giữa các ứng dụng, phiên bản Word và máy in.

Bạn sẽ làm được gì?

Sau phần nền tảng, bạn có thể tự mở kho, đọc tài liệu, tạo nhánh, thay đổi tệp, lưu commit, gửi pull request và hợp nhất thay đổi. Sau phần thực hành trên máy, bạn có thể biết bản nào đang ở máy mình, bản nào đã lên GitHub, kiểm tra trước khi đồng bộ và xử lý các lỗi thường gặp. Các phần sau giúp bạn dùng Issues, Actions, Pages và môi trường đám mây đúng mục đích.

“Thành thạo” trong tài liệu này nghĩa là làm được việc, biết kiểm tra kết quả và biết khi nào cần dừng để bảo vệ dữ liệu. Bạn không cần học hết mọi lệnh Git nâng cao ngay từ đầu.

Đọc theo nhu cầu của bạn

Nhu cầu	Bắt đầu ở đâu?	Kết quả cần đạt
Tôi chưa nhớ GitHub là gì	Chương 1–3	Phân biệt Git, GitHub, tệp và commit
Tôi chỉ muốn thao tác bằng chuột	Chương 4–7	Hoàn thành một pull request bằng web hoặc Desktop
Tôi muốn học lệnh	Chương 8–12	Clone, tạo nhánh, commit, push và xử lý xung đột
Tôi muốn làm dự án thật	Chương 9 và 13–17	Quản lý công việc, phối hợp, kiểm tra và bàn giao
Tôi gặp lỗi hoặc cần tra nhanh	Chương 16 và 18	Chẩn đoán đúng tình trạng trước khi sửa

Hãy bắt đầu bằng trình duyệt. Sau khi hoàn thành bài thực hành DQY đầu tiên, chọn GitHub Desktop nếu muốn dùng giao diện; học terminal khi đã hiểu quy trình. Không cần thực hiện cả ba cách cùng lúc trên cùng một thay đổi.

Quy ước khi đọc và làm theo

- **Làm gì:** thao tác hoặc mục tiêu cụ thể.
- **Vì sao:** lý do cần thao tác đó.
- **Cách làm:** các bước theo thứ tự.
- **Kiểm tra:** dấu hiệu chứng minh đã hoàn tất.
- **Lệnh:** gõ từng dòng trong terminal, nhấn Enter và xem kết quả trước khi sang dòng tiếp theo. Không gõ dấu nhắc $ nếu bạn gặp nó trong nguồn khác.

Các chỗ như YOUR_USERNAME, EMAIL_CUA_BAN, TEN_TEP hoặc SHA_COMMIT là chỗ điền thông tin của bạn, không phải thông tin được tạo sẵn. Ví dụ lệnh dùng URL https://github.com/nix6242/DQY.git vì đó là kho bạn chọn; bạn vẫn phải có quyền thích hợp để đẩy thay đổi. Nếu thao tác trên kho của người khác, dùng fork hoặc xin quyền như chương 6 và 9.

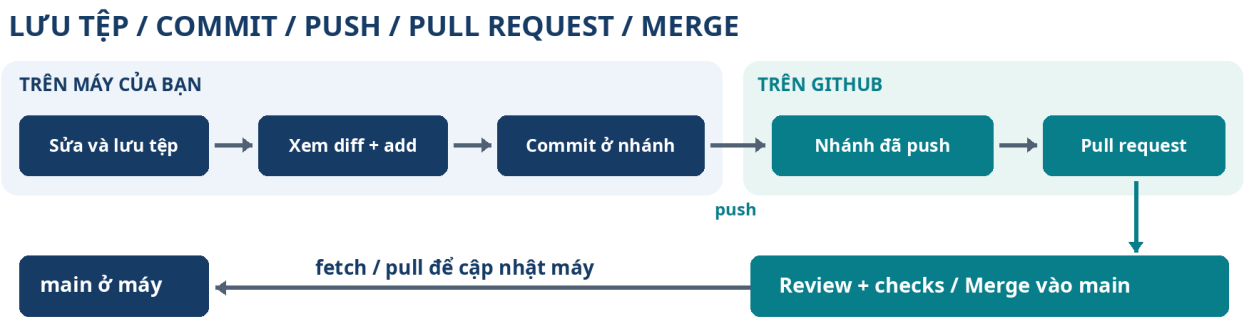
Tài liệu chủ yếu dành cho GitHub.com. GitHub Enterprise của công ty có thể dùng tên miền, quyền và chính sách khác. Giao diện, hạn mức và gói dịch vụ có thể thay đổi; nếu nút không giống ảnh mô tả bằng chữ, tìm theo tên chức năng và xem tài liệu chính thức ở cuối.

DQY đang ở trạng thái nào?

Ở lần kiểm tra ngày 11/10/2026, kho nix6242/DQY chỉ có tệp README.md chứa tiêu đề # DQY. Bản đang có trong môi trường đám mây khớp với nhánh main ở thời điểm đó. Vì chưa có mã ứng dụng, chưa có bước cài thư viện, chạy ứng dụng hay kiểm thử ứng dụng nào để thực hiện.

Các bài trong cẩm nang là **hướng dẫn để bạn thực hành**. Việc tạo tài liệu này không tự tạo nhánh, sửa README, đẩy commit, tạo pull request hoặc xuất bản website trong kho DQY. Hãy kiểm tra trạng thái kho hiện tại trước khi làm vì nó có thể đã thay đổi sau lần kiểm tra.

Sơ đồ quy trình cần nhớ



Làm bằng trình duyệt: sửa và commit diễn ra trực tiếp trên GitHub.
Push gửi commit; merge hợp nhất thay đổi. Một việc không thay cho việc còn lại.

Từ tệp trên máy đến thay đổi được hợp nhất trên GitHub

Trình duyệt có thể thực hiện sửa tệp và commit trực tiếp trên GitHub. Khi làm trên máy, lưu tệp, commit và push là ba việc khác nhau: lưu để giữ nội dung trong tệp; commit để lưu phiên bản vào lịch sử Git; push để gửi lịch sử đó lên GitHub.

Ba thói quen cho buổi học đầu tiên

1. Mỗi lần thực hành chỉ làm một thay đổi nhỏ và kiểm tra kết quả.
2. Trước khi commit, xem đúng kho, đúng nhánh và đúng tệp. Trước khi thay bỏ dữ liệu chưa commit, sao lưu nội dung cần giữ.
3. Ghi lại một câu: “Tôi vừa làm gì, vì sao, kết quả ở đâu?”. Câu đó cũng là nền tảng cho ghi chú commit và mô tả pull request.

Khi học, dùng một thư mục hoặc kho thử nghiệm cho các bài gây xung đột và hoàn tác. Không dùng dự án quan trọng để thử lệnh bạn chưa hiểu.

Chương 1. Hiểu GitHub bằng một dự án thật

Bạn đang học để làm được việc gì?

GitHub giúp lưu dự án, xem lại thay đổi và làm việc cùng người khác. Bạn không cần biết lập trình để bắt đầu. Trong phần này, bạn dùng trình duyệt và kho `nix6242/DQY` để sửa phần giới thiệu, lưu thay đổi và đưa nội dung vào bản chính.

Tại lần kiểm tra trước, DQY chỉ có `README.md`. Nếu hiện có thêm tệp, hãy đọc trước khi sửa. Đây là bài tập để bạn tự thực hiện; tài liệu không tự thay đổi kho.

Git khác GitHub như thế nào?

Git là công cụ quản lý phiên bản. Nó ghi lại các mốc thay đổi của dự án để bạn biết ai sửa gì, xem bản cũ và kết hợp công việc trên các nhánh. Git có thể chạy trên máy tính mà không cần GitHub.

GitHub là dịch vụ lưu kho Git trên Internet, kèm giao diện xem mã, thảo luận, đề nghị thay đổi và quản lý công việc. GitHub không phải Microsoft Word và cũng không tự chạy mọi chương trình bạn tải lên.

Ví dụ, bạn có bản hướng dẫn trên máy và trên GitHub. Git quản lý các mốc lưu; GitHub giúp chia sẻ kho và cộng tác.

Từ thường gặp	Hiểu đơn giản	Ví dụ trong DQY
Repository, repo, kho	Dự án và lịch sử Git của nó	`nix6242/DQY`
File, tệp	Một tài liệu hoặc mã chương trình	`README.md`
Commit	Một mốc thay đổi đã được Git ghi lại	“Bổ sung mục tiêu dự án”
Branch, nhánh	Một dòng công việc có tên riêng	`main`, `docs/gioi-thieu`

Máy của bạn, máy đám mây và GitHub

Local là nơi bạn đang làm việc trực tiếp, thường là máy cá nhân. Trong một phiên làm việc trên máy đám mây, bản kho nằm trên máy đám mây cũng là bản local của phiên đó. **Remote** là kho ở nơi khác, thường là bản trên GitHub. Một máy mới không tự có các tệp, công cụ hoặc khóa đăng nhập của máy cũ.

Khi dùng Git trên máy, sửa tệp và tạo commit mới chỉ lưu ở máy đó. Muốn GitHub nhận commit, bạn cần **push**. Muốn lấy thay đổi từ GitHub, bạn thường **fetch** rồi xem hoặc **pull** theo quy trình thích hợp. Các chương sau sẽ hướng dẫn lệnh. Khi sửa bằng trình duyệt GitHub, commit được tạo ngay trên kho GitHub; bản trên máy vẫn cần cập nhật riêng.

Đừng xem thư mục máy đám mây là nơi lưu vĩnh viễn. Tệp chưa được đưa lên kho hoặc tải về có thể mất khi môi trường bị xóa. Ngược lại, mật khẩu và khóa bí mật không nên đưa vào GitHub để “sao lưu”.

Stage, commit, branch và pull request

Stage là chọn phần thay đổi sẽ đi vào commit tiếp theo khi làm việc bằng Git trên máy. Bạn có thể sửa ba tệp nhưng chỉ chọn hai tệp để commit. Stage không phải là gửi lên GitHub. Trình sửa trên web không bắt bạn thực hiện riêng lệnh `git add`; giao diện sẽ cho bạn xác nhận thay đổi trước khi tạo commit.

Commit lưu trạng thái dự án cùng người tạo, thời điểm, ghi chú và mã nhận diện. Ghi chú nên cụ thể, chẳng hạn `Bổ sung hướng dẫn sử dụng DQY`, thay vì `update`.

Branch giúp sửa việc mới trước khi đưa vào bản chính. Nhánh là tên trỏ tới một dòng lịch sử. `main` thường là nhánh chính; dự án khác có thể dùng tên khác.

Pull request, viết tắt **PR**, là đề nghị đưa thay đổi từ một nhánh vào nhánh khác. PR hiển thị phần khác nhau, trao đổi và kết quả kiểm tra. PR không phải lệnh `git pull`. Mở PR cũng chưa có nghĩa là thay đổi đã vào `main`; phải **merge**, tức hợp nhất, khi đủ điều kiện.

Bạn sẽ thực hành: tạo nhánh → sửa tệp → commit → mở PR → kiểm tra → merge → xác nhận bản chính. Bạn có thể tự hợp nhất PR nếu quyền và quy định kho cho phép.

Chương 2. Chuẩn bị tài khoản và quyền truy cập

Đăng nhập đúng tài khoản

Vì sao cần làm: bạn phải nhận ra tài khoản nào đang hoạt động để tránh sửa nhầm kho hoặc dùng nhầm danh tính.

1. Mở https://github.com bằng trình duyệt. Kiểm tra đúng tên miền trước khi đăng nhập.
2. Nếu chưa có tài khoản, chọn **Sign up**, tạo tài khoản bằng email bạn kiểm soát và xác minh email theo hướng dẫn.
3. Nếu đã có tài khoản, chọn **Sign in** và đăng nhập bằng phương thức bạn đã thiết lập.
4. Bấm ảnh đại diện, mở hồ sơ và kiểm tra username. Trong địa chỉ github.com/nix6242/DQY, nix6242 là chủ sở hữu kho; đó có thể là tài khoản cá nhân hoặc tên tổ chức.
5. Mở https://github.com/nix6242/DQY. Nếu không thấy kho, kiểm tra tài khoản, địa chỉ và quyền truy cập trước khi kết luận kho bị xóa.

Kết quả mong đợi: bạn xem được kho và biết quyền của mình. Xem kho public không đồng nghĩa được ghi. Thiếu quyền sửa thì xin cấp quyền hoặc dùng fork; không gửi mật khẩu cho ai.

Giữ email và danh tính nhất quán

Vì sao cần làm: email dùng để nhận thông báo, khôi phục tài khoản và gắn một số commit với hồ sơ GitHub. Email đăng nhập, username và tên tác giả commit là các thông tin khác nhau.

1. Mở ảnh đại diện → **Settings** → phần **Emails**; tên hoặc vị trí mục có thể thay đổi.
2. Kiểm tra email còn dùng được và trạng thái đã xác minh.
3. Nếu muốn giữ email riêng tư, đọc và bật tùy chọn bảo vệ email phù hợp. GitHub cung cấp địa chỉ noreply dành cho tài khoản; hãy lấy địa chỉ thực tế được hiển thị trong Settings.
4. Khi cấu hình Git trên máy ở chương sau, dùng email đã chọn hoặc địa chỉ noreply đó. Đừng tự đoán địa chỉ noreply, vì định dạng có thể phụ thuộc tài khoản.

Kết quả mong đợi: bạn nhận được email cần thiết và biết địa chỉ nào sẽ được dùng cho commit. Đổi cài đặt hiện tại không tự xóa email đã xuất hiện trong lịch sử cũ.

Bật xác thực hai yếu tố và chuẩn bị khôi phục

Vì sao cần làm: mật khẩu bị lộ không nên đủ để người khác chiếm tài khoản. Đồng thời, bạn cần cách đăng nhập lại khi mất điện thoại hoặc thiết bị xác thực.

1. Trong **Settings**, tìm **Password and authentication** hoặc mục bảo mật tương đương.
2. Thiết lập **two-factor authentication**, thường viết là **2FA**, theo các phương thức GitHub cung cấp. Ứng dụng xác thực, passkey hoặc khóa bảo mật là những lựa chọn cần xem xét theo thiết bị bạn có.
3. Làm đủ bước xác nhận, không dừng ở việc quét mã rồi đóng trang.
4. Lưu **recovery codes**, tức mã khôi phục, vào nơi riêng tư mà bạn vẫn truy cập được khi mất điện thoại, chẳng hạn trình quản lý mật khẩu hoặc bản in cất an toàn.
5. Nếu khả dụng, thêm phương thức dự phòng. Kiểm tra bạn biết nơi tìm mã khôi phục mà không đăng xuất tùy tiện để “thử mất tài khoản”.

Kết quả mong đợi: bảo mật đã cấu hình và có phương án dự phòng. Không đưa mã QR xác thực, mã khôi phục, token, mật khẩu hay khóa riêng vào kho, Word hoặc chat. Người hướng dẫn không cần biết giá trị bí mật.

Nếu thuộc tổ chức, làm theo yêu cầu bảo mật và hỏi quản trị viên khi cần quyền.

Phân biệt công khai và riêng tư

Kho **public** cho phép người trên Internet xem nội dung công khai. Kho **private** giới hạn người được quyền truy cập. Kho riêng tư vẫn được chia sẻ với thành viên hoặc ứng dụng mà bạn cấp quyền; vì thế đừng coi nó là két chứa bí mật.

Trước khi tải tệp lên, hãy tự kiểm tra có thông tin cá nhân, dữ liệu khách hàng, khóa API hoặc nội dung bạn không có quyền chia sẻ hay không. Việc đổi sang private hoặc xóa tệp sau đó không bảo đảm thu hồi những bản người khác đã tải về.

Chương 3. Nhận biết giao diện và tạo kho khi cần

Đọc trang kho trước khi bấm nút

Giao diện có thể thay đổi theo thời gian, màn hình, quyền và cấu hình kho. Tìm theo tên chức năng; nếu thiếu nút, kiểm tra quyền và menu mở rộng.

1. Mở kho DQY và nhìn tên chủ sở hữu, tên kho, dấu public hoặc private.
2. Chọn tab **Code** để xem danh sách tệp.
3. Tìm bộ chọn nhánh và kiểm tra nhánh đang xem. Đừng sửa khi chưa biết mình đang ở `main` hay nhánh bài tập.
4. Bấm `README.md` để xem nội dung. Tìm lịch sử của tệp hoặc danh sách **Commits** để biết đã có những lần thay đổi nào.
5. Xem tab **Pull requests** và **Issues**. Issues là nơi ghi việc cần làm hoặc lỗi; PR là nơi đề nghị đưa thay đổi vào một nhánh.

Kết quả mong đợi: bạn xác định được kho, nhánh, tệp và lịch sử. Thiếu quyền quản trị thì Settings có thể không xuất hiện hoặc không cho sửa.

Mục	Dùng để làm gì	Khi nào bạn cần
Code	Xem tệp và nhánh	Đọc, sửa hoặc lấy mã
Issues	Ghi lỗi, câu hỏi, việc cần làm	Theo dõi công việc
Pull requests	Kiểm tra và hợp nhất thay đổi	Trước khi cập nhật nhánh chính
Actions	Xem quy trình tự động và kết quả	Khi kho đã cấu hình kiểm tra

Kho chỉ có README có thể chưa có chương trình, kiểm thử hay Actions; đó là bình thường.

Tạo kho mới cho bài tập khác

Không cần tạo lại DQY. Các bước sau dùng cho dự án riêng, chẳng hạn `hoc-github`.

Vì sao cần làm: tách dự án giúp quản lý lịch sử và quyền rõ ràng.

1. Trong GitHub, tìm nút **New repository**, thường ở menu dấu cộng hoặc trang danh sách kho.
2. Chọn **Owner** đúng tài khoản hoặc tổ chức mà bạn có quyền tạo kho.
3. Đặt tên dễ hiểu, ví dụ `hoc-github`; có thể thêm mô tả ngắn.
4. Chọn **Private** nếu bài tập có thông tin chưa muốn công khai; chọn **Public** nếu bạn chủ động muốn chia sẻ và đã kiểm tra nội dung.
5. Nếu bắt đầu từ kho trống và chưa có dự án trên máy, có thể chọn thêm README để có tệp đầu tiên.
6. Chọn `.gitignore` phù hợp nếu biết loại dự án; nếu chưa biết, có thể thêm sau khi xác định công nghệ.
7. Cân nhắc license như phần tiếp theo, rồi chọn **Create repository**.

Kết quả mong đợi: kho mới có địa chỉ riêng và đúng mức hiển thị. Nếu bạn đã có kho Git trên máy để đẩy lên, tạo kho GitHub trống thường giúp tránh hai lịch sử khởi đầu riêng; chương làm việc trên máy sẽ hướng dẫn tình huống này.

Chọn license có ý thức

License là giấy phép quy định cách người khác được sử dụng nội dung. Kho public không tự cấp quyền sao chép, sửa hay phân phối; xem được tệp không có nghĩa được dùng tùy ý.

1. Xác định nội dung có thuộc quyền của bạn không. Đọc giấy phép của tài nguyên hoặc thư viện đã lấy từ nơi khác.
2. Xác định mục tiêu: dự án riêng, chia sẻ mã nguồn hay tài liệu công khai.
3. Với mã bạn sở hữu, đọc phần mô tả và toàn văn của lựa chọn như MIT, Apache-2.0 hoặc GPL; chúng có điều kiện khác nhau. Không chọn chỉ vì tên quen.
4. Với tài liệu hoặc hình ảnh, cân nhắc giấy phép phù hợp với loại nội dung, chẳng hạn một lựa chọn Creative Commons khi bạn hiểu điều kiện.
5. Nếu làm cho công ty hoặc dự án chung, hỏi người có thẩm quyền trước khi cấp quyền sử dụng. Có thể để việc chọn license lại cho đến khi xác định được quyền và mục tiêu.

Kết quả mong đợi: bạn hiểu lựa chọn và vẫn tuân thủ giấy phép của tài sản người khác.

Chương 4. Viết README, quản lý tài liệu Word và lấy tệp về

README nên trả lời điều gì?

README hiển thị trên trang đầu kho. Hãy viết: dự án là gì, mục tiêu, hiện trạng và cách bắt đầu. Chỉ ghi lệnh chạy đã có và được kiểm tra; kho chưa có chương trình thì nói rõ.

Markdown là văn bản có ký hiệu đơn giản để tạo tiêu đề, danh sách, liên kết và khối mã. Đuôi .md cho biết tệp thường dùng định dạng này; nó không phải tệp Word.

```
# DQY

Kho này dùng để học GitHub và ghi lại hướng dẫn dự án.

## Hiện trạng

- Đã có tài liệu giới thiệu.
- Chưa có ứng dụng để chạy.

## Bắt đầu

Đọc README này và xem lịch sử thay đổi trong GitHub.

## Tài liệu

[Mở hướng dẫn](docs/huong-dan.md)
```

Liên kết ví dụ tới docs/huong-dan.md chỉ hoạt động khi tệp đó tồn tại. Hãy bỏ dòng ấy hoặc tạo tệp tương ứng trước khi xem README là hoàn chỉnh.

1. Dùng một dấu # cho tiêu đề lớn, ## cho phần bên trong.
2. Để một dòng trống giữa tiêu đề, đoạn văn và danh sách để dễ đọc.
3. Dùng - cho từng mục liệt kê và dấu nháy ngược cho tên tệp hoặc lệnh ngắn.
4. Dùng **Preview** trong trình sửa để kiểm tra cách hiển thị.

Kết quả mong đợi: README đọc được trực tiếp trên GitHub và không đưa ra hướng dẫn giả định vượt quá những gì dự án thực sự có.

Tạo tệp hoặc thư mục bằng trình duyệt

Vì sao cần làm: bổ sung tài liệu mà chưa cần cài Git.

1. Chọn nhánh làm việc, ví dụ docs/gioi-thieu, trước khi thêm tệp.
2. Trong tab Code, tìm **Add file** → **Create new file** hoặc chức năng tương đương.
3. Nhập tên docs/huong-dan.md và viết nội dung. Phần trước dấu / là thư mục; GitHub tạo nó cùng tệp.
4. Xem lại tên, nội dung và nhánh, rồi mở hộp **Commit changes**.
5. Ghi chú cụ thể, chẳng hạn Thêm tài liệu bắt đầu sử dụng, và commit vào nhánh đang làm.

Kết quả mong đợi: tệp xuất hiện trong thư mục docs của đúng nhánh. Git không quản lý thư mục rỗng theo cách thông thường; tạo một tệp bên trong là cách tạo thư mục hữu ích.

Đưa tài liệu Word lên GitHub

Vì sao cần làm: bạn có thể lưu và chia sẻ bản .docx cùng dự án. Nhưng GitHub thường không hiển thị nội dung và chênh lệch của Word thuận tiện như Markdown; muốn sửa nội dung, hãy dùng Word hoặc ứng dụng tương thích.

1. Mở Word, tạo tài liệu và lưu thành Huong-dan-DQY.docx trên máy. Dùng tên dễ nhận biết, tránh ghi dữ liệu riêng tư trong tên tệp.
2. Đọc lại tài liệu, kiểm tra hình ảnh, thông tin cá nhân, bình luận và thay đổi đang theo dõi nếu có. Chỉ tải nội dung phù hợp với người có thể truy cập kho.
3. Trên GitHub, chọn nhánh làm việc rồi **Add file** → **Upload files**. Có thể mở thư mục docs trước nếu muốn đặt tài liệu ở đó.
4. Kéo tệp vào vùng tải hoặc chọn tệp từ máy. Đợi tải xong và xem đúng tên.
5. Tạo commit có ghi chú rõ, sau đó dùng PR nếu bạn đang làm trên nhánh riêng.

Kết quả mong đợi: người có quyền tải được tệp .docx. Upload bằng trình duyệt giới hạn mỗi tệp 25 MiB. Với tệp lớn hơn, xem chương quản lý tệp lớn; đừng đổi đuôi tùy tiện.

Khi cập nhật Word, tải bản mới nhất, sửa rồi tải lại đúng vị trí. Git khó tự hợp nhất hai bản .docx do hai người sửa; hãy thống nhất người đang làm. Markdown phù hợp với hướng dẫn đọc trên GitHub; Word phù hợp với bản cần in.

Tải một tệp hoặc tải cả kho dạng ZIP

Vì sao cần làm: tải về giúp đọc ngoại tuyến hoặc lấy tài liệu mà chưa cài Git.

1. Để lấy một tệp Word, mở tệp trong kho rồi tìm chức năng tải xuống hoặc **View raw** theo giao diện đang thấy. Tệp văn bản có thể mở nội dung raw; dùng chức năng lưu của trình duyệt khi cần.
2. Để lấy cả trạng thái của một nhánh, chọn đúng nhánh trong tab Code.
3. Mở **Code** → **Download ZIP** nếu chức năng này có sẵn.
4. Giải nén vào thư mục mới và kiểm tra tệp bên trong. Đừng giải nén đè lên bản làm việc đang có thay đổi.

Kết quả mong đợi: bạn nhận tệp của nhánh đã chọn. ZIP không có thư mục .git và lịch sử Git nên không dùng như clone để push. Hãy đọc hướng dẫn, kiểm tra nội dung trước khi chạy mã.

Chương 5. Thực hành DQY: từ nhánh mới đến PR đã hợp nhất

Mục tiêu và điểm kiểm tra ban đầu

Bài tập bổ sung phần giới thiệu bằng trình duyệt, không cần token hay phần mềm Git. Bạn sẽ học một vòng làm việc đầy đủ.

1. Mở https://github.com/nix6242/DQY và xác nhận đúng tài khoản.
2. Trong Code, chọn main; nếu kho dùng nhánh chính tên khác, thay main bằng tên thực tế trong các bước.
3. Đọc README đang có. Nếu đã có nội dung quan trọng, giữ lại và chỉ bổ sung phần phù hợp.
4. Kiểm tra bạn có quyền tạo nhánh và sửa. Nếu thiếu, dừng phần ghi của bài tập và dùng quy trình fork ở chương tiếp theo hoặc xin đúng quyền.

Kết quả mong đợi: bạn biết điểm xuất phát và tránh ghi đè công việc có sẵn.

Bước 1: tạo nhánh docs/gioi-thieu

Lý do: nhánh riêng cho phép xem lại thay đổi trước khi cập nhật main.

1. Khi đang ở main, mở bộ chọn nhánh.
2. Nhập docs/gioi-thieu vào ô tìm hoặc tạo nhánh.
3. Chọn chức năng tạo nhánh mới từ main. Nếu nhánh này đã tồn tại, kiểm tra nội dung; dùng tên khác như docs/gioi-thieu-2 cho bài tập mới thay vì xóa nhánh cũ.
4. Kiểm tra bộ chọn nhánh đã hiển thị nhánh mới.

Kết quả mong đợi: bạn đang xem docs/gioi-thieu; tại thời điểm vừa tạo, các tệp giống main. Tạo nhánh chưa tự sửa README.

Bước 2: sửa README

Lý do: một thay đổi nhỏ và nhìn thấy được giúp bạn hiểu toàn bộ quy trình.

1. Trên nhánh mới, bấm README.md.
2. Tìm biểu tượng bút chì hoặc **Edit this file**. Nếu không thấy, kiểm tra nhánh và quyền ghi.
3. Giữ lại nội dung hữu ích; thêm đoạn dưới đây nếu phù hợp với tình trạng hiện tại.

```
## Mục tiêu

Tôi dùng DQY để học GitHub và ghi lại quá trình xây dựng dự án.

## Hiện trạng

Kho hiện có tài liệu giới thiệu. Tôi sẽ bổ sung hướng dẫn chạy
khi dự án có mã nguồn và đã được kiểm tra.
```

4. Chọn Preview và đọc lại câu chữ. Tránh ghi thông tin chưa đúng với kho thực tế.
5. Kiểm tra lần nữa bạn đang sửa trên nhánh bài tập.

Kết quả mong đợi: bản xem trước có hai phần rõ ràng; main chưa thay đổi.

Bước 3: tạo commit

Lý do: commit ghi nhận thay đổi của nhánh để có thể so sánh và mở PR.

1. Bấm **Commit changes** hoặc nút hoàn tất chỉnh sửa tương đương.
2. Nhập ghi chú `Bổ sung mục tiêu và hiện trạng DQY`.
3. Trong lựa chọn nhánh, commit vào nhánh bài tập đang mở. Nếu đã tạo nhánh ở bước 1, bạn không cần tạo thêm một nhánh nữa trong hộp này.
4. Đọc phần thay đổi nếu giao diện cung cấp rồi xác nhận.
5. Mở lại README của nhánh bài tập; sau đó chuyển sang main để so sánh.

Kết quả mong đợi: nhánh bài tập có nội dung mới và một commit mới; main vẫn có nội dung trước đó.

Bước 4: mở pull request

Lý do: PR gom nội dung cần xem xét và xác định rõ thay đổi sẽ đi từ đâu đến đâu.

1. Bấm **Compare & pull request** nếu GitHub gợi ý. Nếu không có, vào **Pull requests → New pull request**.
2. Kiểm tra **base** là main, tức nhánh nhận thay đổi.
3. Kiểm tra **compare** hoặc head là `docs/gioi-thieu`, tức nhánh mang thay đổi. Không đảo hai nhánh.
4. Đặt tiêu đề `Bổ sung phần giới thiệu DQY`.
5. Viết mô tả ngắn: `Thêm mục tiêu và hiện trạng để người mới hiểu kho. Đã xem Preview; chưa có ứng dụng để kiểm thử.`
6. Xem phần chênh lệch rồi bấm **Create pull request**. Với bài tập sẵn sàng kiểm tra, tạo PR thông thường; draft PR phù hợp với công việc còn đang làm.

Kết quả mong đợi: PR có trang riêng và trạng thái mở. Nội dung vẫn chưa vào main.

Bước 5: xem lại và hợp nhất

Lý do: đây là điểm bạn phát hiện lỗi trước khi cập nhật bản chính.

1. Trong PR, mở **Files changed** và đọc các dòng thêm hoặc xóa. Với bài này chỉ nên có thay đổi README dự kiến.
2. Xem tab trao đổi và phần kiểm tra. Kho đơn giản có thể chưa cấu hình checks; đừng diễn giải thành “mọi kiểm thử đã chạy”.
3. Nếu cần sửa, quay lại nhánh bài tập, sửa và tạo commit mới. PR đang mở sẽ cập nhật theo các commit của nhánh đó.
4. Khi nội dung đúng và GitHub cho phép, chọn **Merge pull request** hoặc phương thức hợp nhất dự án cho phép, rồi xác nhận.
5. Nếu nút merge bị khóa vì cần review, checks hay có conflict, đọc lý do và xử lý theo quy định. Không bỏ qua điều kiện chỉ để hoàn thành bài tập.

Kết quả mong đợi: PR chuyển sang **Merged** khi hợp nhất thành công. Đóng PR bằng nút Close mà chưa merge không đưa thay đổi vào main.

Bước 6: kiểm tra bản chính và dọn nhánh

1. Quay lại tab Code, chọn main và mở README.
2. Xác nhận nội dung mới xuất hiện; xem lịch sử để thấy lần cập nhật.
3. Nếu bài tập đã xong và nhánh không còn dùng, có thể chọn **Delete branch** trên PR. Xóa nhánh đã merge không xóa nội dung đã vào main.
4. Nếu bạn có bản clone trên máy hoặc đám mây, cập nhật bản đó theo hướng dẫn Git ở chương sau. Web và máy không tự đồng bộ tức thì.

Kết quả mong đợi: bạn chỉ được nhánh, PR và nội dung đã có trong main.

Chương 6. Chọn clone, fork hay Download ZIP

Ba lựa chọn giải quyết ba nhu cầu

Lựa chọn	Bạn nhận được gì	Dùng khi nào
Download ZIP	Bản tệp của trạng thái đã chọn	Đọc hoặc lấy tài liệu nhanh
Clone	Bản kho Git trên máy, có lịch sử	Làm việc bằng Git trên máy
Fork	Kho riêng trên GitHub có quan hệ với kho gốc	Đóng góp khi không ghi được vào kho gốc

Clone không tạo kho trên GitHub; fork không tự tải về máy. Bạn có thể fork rồi clone fork, hoặc clone kho mình có quyền mà không cần fork.

Clone khi muốn làm việc trên máy

Lý do: clone đưa lịch sử Git và cấu hình remote về máy để bạn có thể commit, xem nhánh và đồng bộ bằng Git.

1. Cài và cấu hình Git theo chương làm việc trên máy trước khi dùng lệnh.
2. Trong kho, bấm Code và chọn địa chỉ HTTPS hoặc SSH phù hợp với phương thức đăng nhập đã thiết lập.
3. Sao chép địa chỉ. Với HTTPS của DQY, địa chỉ thường là `https://github.com/nix6242/DQY.git`.
4. Clone vào thư mục mới, không ghi đè thư mục chứa dự án khác. Các chương sau có lệnh đầy đủ và cách xác nhận.
5. Kiểm tra remote và nhánh trước khi sửa hoặc push.

Kết quả mong đợi: máy có một bản kho làm việc. Muốn push cần quyền ghi và phương thức xác thực phù hợp; việc clone được kho public không cấp quyền push.

Fork khi muốn đóng góp vào kho người khác

Lý do: bạn giữ một kho thuộc quyền mình, tạo thay đổi ở đó và gửi PR sang kho gốc.

1. Đọc hướng dẫn đóng góp của kho gốc nếu có.
2. Bấm Fork và chọn tài khoản được phép nhận fork. Một số kho hoặc tổ chức có thể giới hạn khả năng fork.
3. Kiểm tra địa chỉ kho mới có tên chủ sở hữu của bạn; đừng nhầm với kho gốc.
4. Tạo nhánh làm việc trong fork, sửa và commit.
5. Mở PR về kho gốc, kiểm tra cả tên kho lẫn base và compare.
6. Chờ người có quyền ở kho gốc review và quyết định merge. Việc bạn có quyền ghi trong fork không cho bạn quyền tự merge ở kho gốc.

Kết quả mong đợi: PR gửi đúng nơi. Fork cần được cập nhật khi kho gốc thay đổi; đừng mặc định mọi nhánh tự đồng bộ.

Tự kiểm tra trước khi học phần tiếp theo

Bạn đã hiểu nếu xác định được kho, nhánh, nơi lưu commit, trạng thái merge và người được xem tệp. Nếu còn lẫn lộn, làm lại bài README trong nhánh mới. Hãy nhớ mục đích và kiểm tra kết quả sau mỗi bước.

Chương 7. Dùng GitHub Desktop: làm việc trên máy bằng giao diện

7.1. Hiểu luồng làm việc trước khi bấm nút

Bạn sẽ làm gì? Tải kho DQY về máy, sửa một tài liệu trên nhánh riêng, lưu thành commit, đưa nhánh lên GitHub rồi đề nghị nhập thay đổi vào `main`.

Vì sao cần học? Khi dự án có nhiều tệp, chỉnh sửa trên máy thường thuận tiện hơn sửa từng tệp trên website. GitHub Desktop giúp bạn nhìn thấy thay đổi và thực hiện nhiều thao tác Git mà chưa phải nhớ lệnh.

Luồng cần nhớ là: **clone → tạo nhánh → sửa → xem thay đổi → commit → push → pull request → merge**. Commit lưu lịch sử trên máy bạn. Push gửi các commit đó lên GitHub. Tạo pull request, gọi tắt là PR, là đề nghị xem xét và nhập một nhánh vào nhánh khác. Merge thực hiện việc nhập đó.

GitHub Desktop chính thức dành cho Windows và macOS. Trên Linux, ưu tiên Git CLI ở Chương 8. Desktop thực hiện các chức năng riêng mà không cần bạn cài Git CLI; muốn chạy lệnh `git` trong terminal, kiểm tra và cài Git CLI nếu còn thiếu.

7.2. Cài đặt, đăng nhập và tải DQY về máy

Các bước:

1. Mở `https://desktop.github.com/` và tải phiên bản phù hợp hệ điều hành. Chạy bộ cài theo hướng dẫn.
2. Mở GitHub Desktop. Chọn đăng nhập GitHub.com. Ứng dụng sẽ chuyển sang trình duyệt để bạn đăng nhập và cho phép Desktop kết nối với tài khoản. Kiểm tra đúng website GitHub trước khi nhập thông tin.
3. Nếu bạn có nhiều tài khoản GitHub, nhìn lại tài khoản đang đăng nhập trước khi cấp quyền. Sau bước này trình duyệt có thể hỏi mở lại GitHub Desktop; đồng ý để tiếp tục.
4. Trong phần cấu hình Git, đặt tên người viết commit và email phù hợp. Tên có thể là tên thật hoặc tên bạn muốn hiển thị. Email nên là email đã xác minh trên GitHub hoặc email riêng tư `noreply` do GitHub cung cấp. Cách tìm nằm ở mục 8.2.
5. Chọn **File → Clone repository**. Có thể chọn DQY từ danh sách tài khoản hoặc chọn tab **URL** và nhập `https://github.com/nix6242/DQY.git`.
6. Chọn **Local path** là vị trí lưu trên máy, chẳng hạn thư mục dự án trong Documents. Kiểm tra đường dẫn cuối cùng: đây là bản sao làm việc trên máy, không phải địa chỉ website.
7. Bấm **Clone**, chờ hoàn tất rồi nhìn **Current repository** và **Current branch**. Chúng cần là DQY và `main` nếu kho đang dùng `main` làm nhánh mặc định.

Kết quả cần thấy: Desktop hiển thị kho DQY; thư mục vừa chọn có các tệp của kho. Nếu kho chưa có gì ngoài README, bạn chỉ thấy ít tệp là bình thường. Không cần tạo lại kho trên website.

DQY đã có trên máy thì chọn **File → Add local repository** để tránh tạo nhiều bản và sửa nhầm nơi.

7.3. Bài thực hành: sửa README trên một nhánh

Bạn sẽ làm gì? Thêm một đoạn mục tiêu vào README mà chưa thay đổi trực tiếp nhánh chính.

Vì sao dùng nhánh? Nhánh riêng cho phép bạn sửa và kiểm tra một công việc độc lập. `main` vẫn giữ bản đã được chấp nhận. Cách này hữu ích ngay cả khi chỉ có một người làm.

1. Đảm bảo Desktop đang mở DQY. Chọn `main`, sau đó **Fetch origin** để hỏi GitHub có thay đổi mới hay không. Nếu nút chuyển thành **Pull origin**, bấm để lấy và nhập các thay đổi đó. Nếu đang có sửa dở, xử lý chúng trước khi đổi nhánh.
2. Mở **Current branch** → **New branch**. Đặt tên `docs/huong-dan-dqy`, chọn tạo từ `main` và bấm tạo nhánh.
3. Chọn mở thư mục trong File Explorer hoặc Finder. Mở `README.md` bằng trình soạn thảo văn bản như VS Code. Không dùng Word để sửa tệp `.md`, vì Word tạo tài liệu định dạng riêng.
4. Thêm một đoạn nhỏ, ví dụ `## Mục tiêu` và câu "Tôi dùng DQY để học cách làm việc với GitHub." Lưu tệp trong trình soạn thảo.
5. Quay lại Desktop. Tab **Changes** phải xuất hiện `README.md`. Bấm tên tệp và đọc từng dòng thêm hoặc xóa. Màu xanh thường là dòng thêm, màu đỏ thường là dòng xóa. Đây là phần so sánh, gọi là **diff**.
6. Chỉ đánh dấu tệp thuộc việc này. Nếu có tệp lạ như cấu hình cá nhân hoặc tài liệu riêng, đừng đưa vào commit chỉ vì nó xuất hiện trong danh sách.
7. Nhập **Summary**, ví dụ `docs: bổ sung mục tiêu cho README`. Phần mô tả dài hơn là tùy chọn. Bấm **Commit to docs/huong-dan-dqy**.
8. Mở tab **History** để thấy commit vừa tạo. Đọc lại tên nhánh trước khi đi tiếp.

Kết quả cần thấy: commit xuất hiện trong History của nhánh riêng. Nếu chưa push, GitHub trên trình duyệt chưa nhận commit này. Đóng rồi mở Desktop không làm mất commit đã lưu.

7.4. Đưa nhánh lên GitHub và mở pull request

1. Nếu nhánh chưa từng có trên GitHub, bấm **Publish branch**. Với nhánh đã được xuất bản, nút thường là **Push origin**. Hãy kiểm tra tài khoản và kho đích nếu ứng dụng báo không có quyền.
2. Sau khi gửi thành công, chọn **Create Pull Request** nếu Desktop hiển thị. Bạn cũng có thể mở DQY trên trình duyệt, vào **Pull requests** → **New pull request**.
3. Kiểm tra **base** là `main`, **compare** là `docs/huong-dan-dqy`. Base là nơi sẽ nhận thay đổi; compare là nhánh chứa công việc của bạn.
4. Đặt tiêu đề rõ ràng: "Bổ sung mục tiêu cho README". Trong mô tả, ghi đã sửa gì, vì sao và đã kiểm tra bằng cách nào. Ví dụ: "Thêm mục tiêu học GitHub; đã kiểm tra phần hiển thị Markdown."
5. Mở **Files changed** và đọc lại toàn bộ thay đổi. Nếu chưa xong, có thể tạo **Draft pull request** khi giao diện cho phép. Bản nháp thể hiện công việc chưa sẵn sàng để nhập.
6. Với bài tập một mình và chỉ sửa tài liệu, bạn có thể tự xem xét rồi merge khi không còn vấn đề. Trong nhóm, làm theo người có quyền duyệt và quy định của dự án.

Kết quả cần thấy: PR có số, tiêu đề, nhánh nguồn và nhánh đích. Merge xong, README mới xuất hiện trên `main` của GitHub. Trên máy, chuyển về `main` và lấy cập nhật trước khi tạo công việc tiếp theo.

7.5. Fetch, pull và push khác nhau thế nào?

Thao tác	Hướng dữ liệu	Ý nghĩa thực tế
Fetch	GitHub → máy	Cập nhật thông tin về các nhánh trên GitHub; chưa tự nhập vào nhánh làm việc hiện tại.
Pull	GitHub → máy	Lấy cập nhật rồi kết hợp vào nhánh hiện tại; có thể phát sinh xung đột.
Push	Máy → GitHub	Gửi commit đã tạo trên máy; không gửi riêng một tệp đang sửa mà chưa commit.

Cách nhớ: cập nhật nhánh chính trước việc mới; commit phần đã hoàn thành; push để gửi commit lên GitHub. Có lỗi thì đọc nguyên nhân trước khi thử lại.

Chương 8. Dùng Git bằng dòng lệnh

8.1. Cài Git và làm quen với terminal

Bạn sẽ làm gì? Cài chương trình Git để thao tác bằng lệnh. Git và GitHub là hai thứ khác nhau: Git chạy trên máy để quản lý lịch sử; GitHub là dịch vụ lưu kho và hỗ trợ cộng tác.

Vì sao học? Hướng dẫn dự án và máy đám mây thường dùng dòng lệnh. Bạn có thể dùng CLI cùng Desktop.

Trên **Windows**, tải Git qua https://git-scm.com/downloads/win. Bộ cài Git for Windows có Git Credential Manager. Dùng lựa chọn mặc định nếu chưa có nhu cầu riêng, sau đó mở Git Bash. Đóng và mở lại terminal cũ nếu chưa nhận lệnh.

Trên **macOS**, kiểm tra `git --version` trong Terminal trước. Nếu hệ điều hành mời cài Command Line Tools, làm theo hộp thoại. Bạn cũng có thể yêu cầu bộ công cụ này bằng lệnh dưới đây; đây là một lựa chọn cài đặt, không cần chạy nếu Git đã có:

```
xcode-select --install
```

Nếu đã dùng Homebrew, có thể theo hướng dẫn trên trang tải Git cho macOS; chỉ cần chọn một cách cài.

Trên **Ubuntu/Debian** có quyền quản trị, thường dùng trình quản lý gói:

```
sudo apt update
sudo apt install git
```

Máy đám mây có thể đã cài Git. Linux khác có thể dùng trình quản lý gói khác; kiểm tra trước.

Sau đó xác nhận:

```
git --version
```

Kết quả: Git trả về số phiên bản. Nếu không tìm thấy lệnh, kiểm tra cài đặt và PATH. Chỉ tải bộ cài từ nguồn chính thức.

Trong terminal, mỗi lệnh dưới đây chạy riêng rồi đọc kết quả. Không gõ dấu ba nháy của khung lệnh. Khi nhập lệnh có <...>, thay phần đó bằng thông tin thật; ký hiệu này không phải nội dung cần gõ nguyên văn.

Hộp nhắc nhanh về terminal: Windows có thể mở Git Bash; macOS mở Terminal. `pwd` cho biết thư mục hiện tại, `ls` liệt kê tệp, `cd ..` đi lên thư mục cha. Trong PowerShell, lệnh tương ứng là `Get-Location` và `Get-ChildItem`. Đường dẫn có khoảng trắng cần dấu nháy, ví dụ `cd "Du an GitHub"` khi thư mục đó ở ngay trong thư mục hiện tại. Nếu `git log` mở màn hình cuộn, bấm `q` để thoát. `Ctrl+C` ngắt lệnh đang chạy; nó không tự hoàn tác những thay đổi lệnh đã thực hiện.

Nếu một lệnh Git mở trình soạn thảo để nhập ghi chú, đọc thông điệp trước khi lưu. Với **Vim**, nhấn Esc, gõ `:wq` rồi Enter để lưu và đóng. Với **Nano**, nhấn Ctrl + O, Enter rồi Ctrl + X. Đây là thao tác hoàn tất thông điệp trong trình soạn thảo, không phải lệnh hủy merge hay revert. Các ví dụ có `-m` hoặc `--no-edit` trong tài liệu giúp hạn chế việc mở trình soạn thảo.

8.2. Đặt tên và email cho commit

Bạn sẽ làm gì? Cấu hình danh tính người tạo commit. Đây không phải bước đăng nhập GitHub, cũng không phải mật khẩu.

1. Trên GitHub, mở ảnh đại diện → **Settings** → **Emails**. Kiểm tra email nào đã được xác minh.
2. Nếu muốn giữ email cá nhân riêng tư, bật lựa chọn giữ email riêng tư và **sao chép đúng địa chỉ `noreply` mà GitHub hiển thị**. Đừng đoán địa chỉ từ tên tài khoản; cấu trúc địa chỉ có thể khác nhau giữa các tài khoản.
3. Thay nội dung ví dụ sau bằng tên và địa chỉ bạn chọn:

```
git config --global user.name "Tên của bạn"
git config --global user.email "EMAIL_DA_CHON"
```

4. Kiểm tra từng trường:

```
git config --global --get user.name
git config --global --get user.email
```

Kết quả: tên và email đúng. `--global` áp dụng cho người dùng hiện tại trên máy; bỏ `--global` khi cần cấu hình riêng trong một kho.

Email commit có thể xuất hiện trong lịch sử. Cấu hình mới không tự sửa commit cũ. Trên máy dùng chung, dùng tài khoản hệ điều hành riêng hoặc hỏi người quản trị trước khi lưu đăng nhập.

8.3. Đăng nhập an toàn: đừng dùng mật khẩu tài khoản cho Git HTTPS

Vì sao cần biết? Mật khẩu đăng nhập website GitHub không phải cách xác thực cho các thao tác Git qua HTTPS. Clone kho công khai có thể không cần đăng nhập, nhưng push vào kho thường cần quyền ghi và xác thực.

Lựa chọn thuận tiện là GitHub Desktop hoặc Git Credential Manager, thường có thể mở trình duyệt để xác thực theo cấu hình được hỗ trợ. Git CLI có thể cần thiết lập riêng; việc Desktop đăng nhập thành công không chứng minh mọi terminal đều đã có đúng cách xác thực.

Nếu bạn chủ động dùng GitHub CLI và đã cài công cụ đó, có thể chạy:

```
gh auth login
```

Chọn GitHub.com, HTTPS và cách đăng nhập bằng trình duyệt; đọc các câu hỏi của công cụ. Nếu cần cho Git dùng xác thực của GitHub CLI, làm theo hướng dẫn và có thể dùng:

```
gh auth setup-git
```

Không cần `gh` để dùng Desktop. Máy đám mây có thể đã được cấp xác thực Git; thử thao tác cần thiết trước khi thêm đăng nhập mới.

Nếu dự án bắt buộc dùng personal access token hoặc SSH, làm theo hướng dẫn chính thức và chọn quyền vừa đủ. **Không đặt token trong URL, README, lệnh chia sẻ, ảnh chụp hoặc tài liệu Word.** Không gửi token cho người hỗ trợ qua chat. Token lộ phải được thu hồi; chỉ xóa đoạn văn chứa token không bảo đảm nó đã biến mất khỏi lịch sử.

8.4. Clone DQY và đọc trạng thái

Nếu đã clone bằng Desktop, bạn có thể mở terminal trong thư mục đó và bỏ qua lệnh clone. Nếu học theo CLI từ đầu, chọn thư mục cha để lưu dự án rồi chạy:

```
git clone https://github.com/nix6242/DQY.git
cd DQY
git status
git log --oneline -5
```

Từng lệnh có nghĩa gì? `clone` tải kho và lịch sử; `cd` chuyển vào thư mục DQY; `status` cho biết nhánh hiện tại cùng các thay đổi; `log` xem tối đa năm commit gần nhất theo dạng gọn.

Kết quả cần thấy: Git nhận diện đây là một kho, hiển thị nhánh và trạng thái tệp. Nội dung log phụ thuộc lịch sử thật. Nếu kho có ít commit, số dòng ít hơn năm là bình thường.

“not a git repository” thường nghĩa là đứng sai thư mục. DQY đã tồn tại thì kiểm tra bản có sẵn, không xóa để clone lại. Xem nơi gửi và nhận dữ liệu bằng:

```
git remote -v
```

`origin` cần trỏ đúng DQY. URL chuẩn không nhúng token; không chia sẻ kết quả có thông tin nhạy cảm.

8.5. Một vòng làm việc bằng CLI

Điều kiện: bạn có quyền ghi vào DQY và không có công việc chưa lưu cần giữ ở nhánh hiện tại. Đọc trạng thái trước:

```
git status
git switch main
git pull --ff-only
git switch -c docs/huong-dan-dqy-cli
```

`switch main` chuyển về nhánh chính. `pull --ff-only` chỉ cập nhật khi Git có thể tiến nhánh thẳng tới lịch sử mới, tránh tự tạo merge commit trong bước này. Nếu bị từ chối vì lịch sử đã tách, dừng và kiểm tra; không chuyển ngay sang force hay reset. `switch -c` tạo rồi chuyển sang nhánh công việc mới. Nếu tên nhánh đã tồn tại, chọn tên mới hoặc mở nhánh cũ có chủ đích.

Mở `README.md`, sửa một đoạn nhỏ và lưu. Sau đó:

```
git status
git diff -- README.md
git add README.md
git diff --cached -- README.md
git commit -m "docs: bổ sung hướng dẫn DQY"
git push -u origin docs/huong-dan-dqy-cli
```

Vì sao xem diff hai lần? `git diff` xem phần chưa staged. `git add README.md` đưa phiên bản hiện tại vào vùng chuẩn bị commit, gọi là **staging area**. `git diff --cached` xem phần sẽ commit. Sửa tiếp sau add thì phần mới chưa được staged; muốn commit cả phần đó, add lại.

Chỉ add tên tệp đã kiểm tra. Người mới dễ vô tình đưa thông tin riêng lên GitHub khi dùng `git add .` mà chưa đọc trạng thái.

`commit` lưu trên máy; `push -u` gửi nhánh và thiết lập liên kết với nhánh tương ứng trên `origin`. Những lần push tiếp theo ở nhánh đó thường chỉ cần `git push`.

Kết quả: nhánh xuất hiện trên GitHub. Mở PR với base `main`, compare là nhánh vừa gửi. Thiếu quyền ghi thì dùng fork hoặc lời mời cộng tác; thử push nhiều lần không tạo quyền.

8.6. Đưa một dự án đã có trên máy lên GitHub

Bạn sẽ làm gì? Đưa thư mục dự án thật lên một kho mới, giữ lịch sử Git hiện có nếu thư mục đã dùng Git. Bài này dành cho **kho GitHub trống**, không dùng để thay thế DQY đã có README.

Vì sao phải tạo kho trống? Hai nơi cùng tự tạo commit ban đầu dễ có lịch sử riêng. Khi tạo kho mới trên GitHub, không chọn tạo README, license hoặc `.gitignore`; bạn có thể thêm chúng từ máy sau.

1. Mở terminal trong đúng thư mục dự án. Sao lưu công việc quan trọng, đọc danh sách tệp và kiểm tra bí mật trước khi upload.
2. Chạy `git status`. Nếu đây đã là kho Git, giữ nguyên lịch sử và bỏ qua bước init. Nếu chưa có Git, sau khi xác nhận đúng thư mục, khởi tạo bằng:

```
git init -b main
```

3. Tạo `.gitignore` phù hợp trước khi add. Chỉ add các tệp đã xem xét, bằng tên cụ thể. Ví dụ sau chỉ dùng nếu dự án thật có `README.md` và `.gitignore`:

```
git add README.md .gitignore
git diff --cached
git commit -m "docs: them tai lieu ban dau"
```

Add các tệp dự án cần chia sẻ bằng tên thật **trước bước diff và commit** ở ví dụ trên. Nếu bạn add thêm tệp sau khi đã commit, hãy kiểm tra rồi tạo commit cho phần mới đó trước khi push. Push chỉ gửi commit, không gửi phần mới chỉ được add. Nếu đã có commit và không sửa gì, không cần tạo commit rỗng để tiếp tục.

4. Kiểm tra remote bằng `git remote -v`. Chỉ thêm `origin` nếu tên này chưa có. Thay hai phần viết hoa trong URL sau bằng tài khoản và tên **kho mới trống** của bạn:

```
git remote add origin https://github.com/TEN_TAI_KHOAN/TEN_KHO_MOI.git
git branch --show-current
git push -u origin HEAD
```

`HEAD` ở đây gửi nhánh hiện tại lên GitHub bằng tên nhánh đó. Kiểm tra bạn đang ở nhánh cần gửi. Nếu đã có `origin`, đọc URL và xác nhận mục đích; không ghi đè remote hiện có chỉ để làm theo ví dụ.

Kết quả: GitHub có commit và tệp đã gửi. Lịch sử cũ của kho trên máy được giữ. Nếu remote thực tế không trống hoặc push bị từ chối, dừng chẩn đoán; không force push để vượt lỗi.

8.7. Tùy chọn: đồng bộ một fork với dự án gốc

Chỉ dùng khi `origin` là **fork của bạn**, còn dự án gốc là kho khác. Không cần làm với DQY thuộc chính tài khoản của bạn.

Vì sao cần? Dự án gốc tiếp tục thay đổi, nhưng fork không tự nhận mọi commit mới. Bạn cần cập nhật nhánh chính để công việc mới bắt đầu từ bản gần nhất.

1. Chạy `git status`, bảo đảm không có sửa dở; kiểm tra `git remote -v`. `origin` phải trỏ đúng fork. Nếu chưa có remote `upstream`, thêm URL thật của dự án gốc; ví dụ dưới chỉ phù hợp khi dự án gốc là DQY:

```
git remote add upstream https://github.com/nix6242/DQY.git
```

2. Xác nhận nhánh chính thực tế. Ví dụ sau giả sử cả hai nơi dùng `main`:

```
git switch main
git pull --ff-only origin main
git fetch upstream
git merge --ff-only upstream/main
git push origin main
```

Kết quả: `main` trên máy và fork nhận cập nhật dự án gốc. Nếu `--ff-only` báo lịch sử tách, dừng và kiểm tra các commit riêng. Không tự reset hoặc force push; bạn có thể cần giữ những thay đổi đó bằng một quy trình khác.

Chương 9. Làm việc nhóm và quy trình mỗi ngày

9.1. Bắt đầu một công việc có phạm vi rõ ràng

Bạn sẽ làm gì? Biến một yêu cầu thành thay đổi nhỏ mà người khác có thể hiểu và kiểm tra.

Vì sao cần? PR gồm nhiều việc không liên quan khó duyệt và khó hoàn tác. Mỗi nhánh nên có một mục tiêu rõ.

1. Đọc README, tài liệu đóng góp, quy định nhánh và issue. Có `AGENTS.md` thì kiểm tra hướng dẫn áp dụng.
2. Tạo hoặc chọn issue, ghi hiện trạng, kết quả mong muốn và cách xác nhận hoàn thành theo quy trình nhóm.
3. Chạy `git status`, cập nhật `main`, tạo nhánh riêng.
4. Sửa, kiểm tra, commit và push. Một issue có thể gồm nhiều commit.

Kết quả: biết mục tiêu nhánh và cách kiểm tra. Không mở nhánh mới cho mỗi lần lưu tệp.

9.2. Viết PR để người khác duyệt được

PR trả lời ba ý: vấn đề, thay đổi, cách kiểm tra. Ví dụ DQY: “Bổ sung hướng dẫn commit và push vào README cho người mới. Đã xem Preview và kiểm tra liên kết.”

1. Kiểm tra kho, base và compare.
2. Đọc **Files changed**; bỏ tệp ngoài phạm vi. Có bí mật thì ngừng gửi thêm và xử lý việc lộ.
3. Chưa xong thì dùng Draft PR; xong thì đánh dấu sẵn sàng duyệt.
4. Mời reviewer theo quy định. Trả lời cụ thể; bất đồng thì giải thích bằng yêu cầu hoặc kiểm tra.
5. Sửa trên cùng nhánh, commit và push. PR tự cập nhật, không cần PR mới cho mỗi góp ý.

Kết quả: reviewer đủ thông tin. Review có thể yêu cầu sửa, chấp thuận hoặc nhận xét; nhận xét chưa chắc cho phép merge.

9.3. Checks, merge và nhánh được bảo vệ

Checks là kết quả tác vụ tự động đã được dự án cấu hình. Không có checks không có nghĩa đã kiểm thử; checks xanh cũng không chứng minh mọi hành vi đều đúng.

Trước khi merge, đọc kết quả checks, xác nhận đã giải quyết góp ý và kiểm tra xung đột. Người có quyền thực hiện merge theo quy định dự án. Các lựa chọn thường gặp:

Cách nhập	Điều xảy ra	Khi thường dùng
Merge commit	Giữ commit của nhánh và tạo commit nhập nhánh.	Nhóm muốn giữ hình dạng lịch sử cộng tác.
Squash and merge	Gộp thay đổi của PR thành một commit trên nhánh đích.	Nhóm muốn lịch sử chính gọn theo từng việc.
Rebase and merge	Đưa các commit lên nhánh đích theo lịch sử tuyến tính.	Nhóm đã thống nhất cách quản lý lịch sử này.

Người mới nên theo lựa chọn của dự án. Không tự đổi cách merge hoặc dùng rebase trên nhánh nhiều người đang cùng dùng.

Chủ kho có thể dùng **branch protection** hoặc **rulesets** để yêu cầu PR, review hay checks. Khả năng sử dụng phụ thuộc loại kho, gói và quyền; kiểm tra Settings cùng tài liệu hiện hành, không mặc định mọi kho private đều hỗ trợ giống nhau.

Sau merge, cập nhật `main` và tạo nhánh mới cho việc mới. Chỉ xóa nhánh cũ khi đã merge và không còn ai dùng. Xóa trên GitHub không tự xóa nhánh trên mọi máy.

9.4. Khi bạn không có quyền ghi: dùng fork

Fork tạo kho riêng dưới tài khoản của bạn từ một kho khác. **Clone** tạo bản làm việc trên máy từ một kho. Hai thao tác phục vụ hai mục đích khác nhau.

Nếu dự án cho phép, tạo fork, clone fork, tạo nhánh rồi push vào fork. PR có kho đích là dự án gốc và kho nguồn là fork. DQY của chính bạn thường không cần fork.

Kho private hoặc nội bộ có thể hạn chế fork. Cập nhật fork bằng chức năng đồng bộ hoặc quy trình dự án; hiểu kho nguồn trước khi nhập lịch sử.

9.5. Mời người cộng tác và chọn quyền phù hợp

Vì sao cần làm: mỗi người dùng tài khoản riêng để GitHub ghi nhận đóng góp và kiểm soát quyền. Không chia sẻ mật khẩu, token hoặc khóa SSH để "cùng dùng một tài khoản".

Với kho thuộc tài khoản cá nhân, người có quyền quản lý có thể làm như sau:

1. Mở đúng kho → **Settings**, tìm **Collaborators**, **Manage access** hoặc mục truy cập tương đương. Tên và vị trí có thể khác; nếu không thấy, kiểm tra quyền của bạn.
2. Chọn **Add people**, tìm đúng username hoặc email theo giao diện. Đối chiếu hồ sơ để tránh mời nhầm người có tên gần giống.
3. Đọc quyền được cấp trước khi gửi lời mời. Kho cá nhân có cách cấp quyền khác kho tổ chức; đừng mặc định mọi kho đều có đủ lựa chọn vai trò.
4. Người được mời đăng nhập tài khoản của họ và chấp nhận lời mời từ GitHub. Gửi lời mời chưa đồng nghĩa họ đã có quyền làm việc.
5. Kiểm tra danh sách truy cập và nhờ người đó mở kho. Thống nhất quy trình nhánh → PR, thay vì để mọi người sửa trực tiếp main.

Kết quả mong đợi: đúng người truy cập bằng danh tính riêng và biết phần việc. Với kho public, lời mời cộng tác cấp quyền làm việc; quyền xem vốn đã công khai.

Với **Organization**, quyền có thể được cấp qua thành viên, **Teams** và vai trò kho. Người quản trị cần xem settings hiện có, chính sách tổ chức và các tính năng của gói đang dùng. Chọn quyền thấp nhất đủ làm việc; vai trò quản trị chỉ dành cho người cần quản lý kho. Không mặc định một người đã vào tổ chức sẽ xem được mọi kho riêng tư.

Khi công việc kết thúc, người có thẩm quyền rà soát và thu hồi quyền không còn cần. Với tổ chức, kiểm tra cả quyền qua team hoặc vai trò khác. Thu hồi quyền không xóa được bản clone hay tài liệu đã tải về máy người đó.

9.6. Tìm nội dung và nhận thông báo đúng việc

Vì sao cần làm: biết tìm tệp và lịch sử giúp giải thích thay đổi; thông báo phù hợp giúp bạn nhận ra PR đang chờ mình review.

1. Trong tab Code, chọn đúng nhánh rồi dùng **Go to file** hoặc chức năng tìm tệp theo tên. Khi cần tìm trong nội dung, dùng ô tìm kiếm GitHub và giới hạn kho, chẳng hạn `repo:nix6242/DQY README`.
2. Mở tệp và tìm **History** để xem các commit liên quan. **Blame**, nếu có, giúp xem dòng hiện tại gắn với commit nào; đó không phải kết luận về người gây lỗi.
3. Khi gửi dẫn chứng, dùng **permalink** tới phiên bản theo commit. Link theo nhánh có thể hiển thị nội dung mới sau lần sửa tiếp theo, còn permalink giữ đúng phiên bản đang thảo luận.
4. Trong Issues hoặc Pull requests, dùng bộ lọc. Ví dụ `is:issue is:open` tìm issue đang mở; `is:pr is:open review-requested:@me` tìm PR đang mở yêu cầu bạn review, trong phạm vi và quyền truy cập hiện tại.

5. Phân biệt **Star** để đánh dấu kho quan tâm với **Watch** để quản lý thông báo. Chọn mức Watch hoặc tùy chỉnh phù hợp; đừng bật mọi hoạt động cho quá nhiều kho.
6. Kiểm tra **Notifications** và settings thông báo email/web. Theo dõi các cuộc thảo luận mình tham gia hoặc được nhắc tên; trong nhóm, thống nhất ai review và cách nhắc khi chờ lâu.

Kết quả mong đợi: bạn tìm đúng nội dung, gửi đúng phiên bản và xem yêu cầu review. Đánh dấu đã đọc không hoàn thành công việc trong issue hoặc PR.

Chương 10. Xử lý xung đột mà không mất công việc

10.1. Xung đột là gì?

Git thường tự ghép thay đổi ở những phần khác nhau. Xung đột xuất hiện khi Git không thể tự quyết định kết quả, chẳng hạn hai nhánh sửa cùng đoạn README, hoặc một nhánh xóa tệp mà nhánh kia sửa tệp đó.

Vì sao cần xử lý bằng hiểu biết? Chọn "giữ của tôi" hoặc "giữ của họ" có thể làm mất yêu cầu của người còn lại. Bạn cần tạo nội dung cuối cùng đúng về ý nghĩa, không chỉ làm thông báo lỗi biến mất.

10.2. Quy trình thực hành an toàn bằng CLI

Giả sử bạn đang làm trên một nhánh tài liệu và cần nhập `main` mới nhất vào nhánh đó. Trước tiên lưu hoặc xử lý công việc chưa commit; kiểm tra trạng thái không còn thay đổi dở có thể bị trộn vào quá trình merge. Không dùng hướng dẫn này khi chưa biết mình đang ở nhánh nào.

```
git status
git branch --show-current
git fetch origin
git merge --no-edit origin/main
```

Chỉ chạy lệnh merge sau khi đã xác nhận nhánh hiện tại là nhánh công việc cần cập nhật. Nếu Git báo xung đột, đọc danh sách tệp và chạy `git status` để biết trạng thái. Một tệp văn bản có thể xuất hiện dấu như sau:

```
<<<<<<< HEAD
Hướng dẫn đang có trên nhánh công việc.
=======
Hướng dẫn được cập nhật trên nhánh main.
>>>>>>> origin/main
```

Trong tình huống merge này, phần giữa `<<<<<<< HEAD` và `=======` thuộc nhánh hiện tại; phần sau `=======` thuộc bên đang được nhập. Ý nghĩa "ours/theirs" có thể gây nhầm trong thao tác khác như rebase, nên hãy đọc nội dung và tên nhánh thực tế.

1. Mở tệp bằng trình soạn thảo. Đọc cả hai phần và hỏi người liên quan nếu yêu cầu mâu thuẫn.
2. Viết lại đoạn cuối cùng cần giữ: có thể giữ một bên, kết hợp cả hai hoặc viết cách diễn đạt mới.
3. Xóa các dòng đánh dấu `<<<<<<<`, `=======`, `>>>>>>>` của vùng xung đột sau khi đã giải quyết. Nếu có nhiều vùng, xử lý từng vùng.
4. Lưu tệp, đọc lại toàn bộ đoạn và chạy kiểm tra phù hợp. Với Markdown, xem Preview để phát hiện lỗi trình bày.
5. Đánh dấu từng tệp đã giải quyết, kiểm tra phần chuẩn bị commit rồi hoàn tất merge:

```
git add README.md
git diff --cached -- README.md
git status
git commit -m "Merge main vao nhanh tai lieu"
git push
```

Chỉ commit khi mọi xung đột đã được xử lý. Không copy một chuỗi lệnh rồi chạy hết nếu trạng thái chưa đúng.

Kết quả cần thấy: Git không còn liệt kê tệp chưa hợp nhất, commit merge hoàn tất và PR cập nhật sau push. Hãy kiểm tra lại nội dung, vì hết dấu xung đột chưa bảo đảm tài liệu hay chương trình đúng.

10.3. Khi muốn dừng merge hoặc gặp tệp Word

Nếu đang có **merge chưa hoàn tất** và muốn quay về trạng thái trước merge, dùng:

```
git merge --abort
git status
```

Lệnh abort dành cho merge đang diễn ra, không dùng để “hoàn tác mọi việc”. Hãy sao lưu bên ngoài kho phần giải quyết xung đột mà bạn muốn giữ trước khi abort. Nó cũng có thể không khôi phục hoàn hảo trạng thái nếu bạn bắt đầu merge khi đã có sửa dở; đó là lý do phải kiểm tra trạng thái sạch trước.

Tệp .docx, hình ảnh và nhiều tệp nhị phân không có các dòng văn bản để Git ghép như README. Với xung đột Word, giữ bản sao cần thiết, mở hai phiên bản trong Word, so sánh nội dung và tạo một phiên bản cuối đã thống nhất. Đưa tệp cuối đó vào kho và đánh dấu đã giải quyết. Đừng chọn một bên chỉ dựa vào tên tệp.

Một số xung đột đơn giản có thể giải quyết ngay trên GitHub qua **Resolve conflicts**; một số cần làm trên máy. Nếu nút bị vô hiệu hóa, đọc lý do và dùng quy trình phù hợp, không cố đổi quyền hoặc dùng force để vượt qua.

Chương 11. Hoàn tác và cất công việc dở một cách an toàn

11.1. Chọn thao tác theo vị trí của thay đổi

Bạn sẽ làm gì? Xác định thay đổi đang ở tệp chưa commit, vùng staging hay lịch sử đã chia sẻ trước khi hoàn tác.

Vì sao cần? Git có thể giữ commit rất tốt, nhưng những thay đổi chưa commit không có cùng mức bảo vệ. Một lệnh khôi phục tệp có thể xóa công việc chưa lưu vào lịch sử.

Luôn bắt đầu bằng:

```
git status
git diff
git diff --cached
```

Nếu phần sửa có giá trị, sao chép tệp ra thư mục an toàn ngoài kho hoặc tạo commit phù hợp trước. Đừng dựa vào khả năng “sẽ tìm lại được” khi chưa biết thay đổi đã được lưu ở đâu.

Tình huống	Thao tác phù hợp	Điều cần nhớ
Add nhầm tệp, chưa commit	Bỏ tệp khỏi staging.	Giữ nội dung tệp đang sửa.
Muốn bỏ phần sửa chưa commit	Khôi phục tệp sau khi sao lưu.	Có thể mất phần sửa chưa được lưu.
Commit đã chia sẻ cần đảo ngược	Tạo commit revert.	Giữ lịch sử để nhóm theo dõi.
Muốn tạm cất việc dở	Stash có kiểm tra.	Stash thường chỉ nằm trên máy hiện tại.

11.2. Bỏ staging và khôi phục tệp là hai việc khác nhau

Nếu đã add nhầm README nhưng muốn giữ nội dung để sửa tiếp:

```
git restore --staged README.md
git status
```

Kết quả: README không còn trong phần chuẩn bị commit, nhưng phần sửa vẫn ở tệp làm việc. Đây không phải lệnh xóa nội dung bạn vừa viết.

Nếu muốn bỏ phần sửa chưa commit trong README, hãy sao lưu và đọc diff trước. Với trường hợp đã bỏ staging rồi, lệnh sau đưa tệp làm việc về bản trong staging, thường là bản commit hiện tại:

```
git restore README.md
```

Cảnh báo cần hiểu: lệnh này ghi đè phần sửa chưa staged trong tệp. Nó không cất phần bị bỏ vào thùng rác của hệ điều hành và không tự tạo commit để khôi phục. Nếu tệp vẫn có nội dung staged, bản được khôi phục sẽ dựa vào staging chứ không nhất thiết là bản `HEAD`. Vì vậy hãy dùng quy trình bỏ staging, kiểm tra rồi mới restore khi mục tiêu là bỏ toàn bộ phần sửa của tệp đã được theo dõi.

Chỉ dùng tên tệp cụ thể. Tệp mới chưa được Git theo dõi không được xử lý giống tệp đã có trong lịch sử. Không dùng lệnh xóa hàng loạt để làm “working tree sạch” khi chưa xem từng tệp.

11.3. Revert một commit đã đưa lên GitHub

Bạn sẽ làm gì? Tạo một commit mới đảo tác dụng của commit cũ. Cách này giúp người khác vẫn hiểu đã có thay đổi gì và vì sao bị đảo ngược.

1. Thống nhất việc hoàn tác với nhóm nếu thay đổi đang được dùng.
2. Cập nhật `main` khi trạng thái sạch và tạo nhánh riêng cho việc hoàn tác.

3. Đọc lịch sử để xác định đúng mã commit. Mã thật nằm ở máy hoặc trang GitHub, không lấy một mã ví dụ từ tài liệu.

```
git log --oneline
```

4. Với một commit thông thường, chạy lệnh sau sau khi thay SHA_COMMIT bằng mã commit thật vừa chọn. --no-edit dùng thông điệp mặc định, tránh mở trình soạn thảo:

```
git revert --no-edit SHA_COMMIT
```

5. Đọc thay đổi mới, chạy kiểm tra cần thiết, push nhánh và mở PR.

Kết quả: lịch sử có commit mới ghi lại việc hoàn tác. Revert có thể gặp xung đột nếu nhiều thay đổi sau đó phụ thuộc đoạn bị đảo. Revert một merge commit cần lựa chọn nhánh chính của merge; đây là trường hợp nâng cao, nên làm cùng người có kinh nghiệm thay vì đoán tham số.

Nếu revert đang dừng vì xung đột, có hai lựa chọn. Muốn tiếp tục: giải quyết nội dung, git add từng tệp rồi chạy git revert --continue; nếu trình soạn thảo mở, lưu và đóng theo công cụ đang dùng. Muốn hủy quá trình revert: sao lưu phần xử lý cần giữ rồi chạy git revert --abort. Đây là trạng thái revert, không dùng git merge --abort thay thế. Chỉ chạy lệnh khi git status xác nhận đúng quá trình đang diễn ra.

Không dùng reset --hard hay force push để thay thế revert trên nhánh đã chia sẻ. Các thao tác sửa lịch sử cần hiểu rõ ai đang sử dụng lịch sử đó và cần một kế hoạch riêng.

11.4. Stash: tạm cất thay đổi chưa hoàn tất

Vì sao dùng? Bạn đang viết tài liệu dở nhưng cần chuyển sang một việc khác. Stash cất một số thay đổi làm việc để bạn lấy lại sau; nó không thay thế việc push bản đã hoàn thành.

Đọc trạng thái trước và kiểm tra không có thông tin nhạy cảm sắp được đưa vào stash. Ví dụ:

```
git status
git stash push -u -m "Dang viet huong dan DQY"
git stash list
git status
```

-u bao gồm tệp chưa được theo dõi, nhưng **không bao gồm tệp bị ignore**. Stash mặc định không có -u cũng không cất những tệp mới chưa được theo dõi. Vì vậy phải kiểm tra trạng thái sau lệnh; đừng mặc định mọi thứ đã được cất.

Khi quay về nhánh phù hợp và trạng thái cho phép, xem stash rồi áp dụng:

```
git stash show --stat "stash@{0}"
git stash apply "stash@{0}"
git status
```

stash@{0} là stash mới nhất tại thời điểm chạy; nếu đã tạo thêm stash thì vị trí có thể thay đổi. Đối chiếu git stash list để chọn đúng bản. Dùng apply giúp bản stash còn lại trong danh sách trong khi bạn kiểm tra kết quả.

Sau khi đã xác nhận khôi phục đúng, lưu thay đổi cần giữ và kiểm tra lại danh sách, mới xóa đúng stash không còn cần:

```
git stash list
git stash drop "stash@{0}"
```

Nếu apply gặp xung đột, giải quyết và kiểm tra trước khi drop. Stash thường ở kho trên máy hiện tại, không tự được push lên GitHub. Khi đổi máy hoặc mất thư mục kho, đừng trông chờ stash xuất hiện ở máy khác.

Chương 12. Sắp xếp kho, quản lý tệp Word và dùng .gitignore

12.1. Một cấu trúc dễ hiểu cho DQY

Bạn sẽ làm gì? Sắp xếp tệp theo mục đích để người khác mở kho là biết bắt đầu từ đâu.

Vì sao cần? GitHub không tự hiểu tệp nào là hướng dẫn, mã nguồn hay bản nháp. Cấu trúc rõ và README tốt giảm việc phải hỏi lại. Không cần tạo thư mục rỗng hoặc sao chép cấu trúc phức tạp khi dự án mới có một tài liệu.

Ví dụ, khi DQY đã có đủ nội dung tương ứng, có thể tổ chức:

```
DQY/
  README.md
  .gitignore
  docs/
    huong-dan-su-dung.md
    Huong-dan-GitHub.docx
  assets/
    anh-minh-hoa.png
  src/
    ...ma-nguon-khi-du-an-co-ung-dung...
```

`README.md` giới thiệu mục tiêu, cách chạy nếu có ứng dụng và liên kết tài liệu. `docs` chứa tài liệu dài. `assets` chứa hình cần dùng. `src` chỉ cần khi có mã nguồn phù hợp. Đây là gợi ý, không phải yêu cầu của GitHub.

Đặt tên rõ và ổn định. Nếu nhóm dùng nhiều hệ điều hành, tránh tên chỉ khác chữ hoa/thường và tránh các ký tự gây khó dùng trong đường dẫn. Tên có dấu vẫn có thể lưu, nhưng tên ngắn, nhất quán thường thuận tiện khi gõ lệnh.

12.2. Lưu tài liệu Word thực tế

GitHub lưu được `.docx`, nhưng việc so sánh từng dòng và xem thay đổi trực tiếp thường thuận tiện hơn với Markdown. Nếu hướng dẫn cần in hoặc trình bày trang đẹp, Word phù hợp; nếu cần cập nhật thường xuyên và review cùng nhóm, có thể dùng Markdown làm bản nguồn rồi xuất Word khi cần.

Quy trình khi dùng Word:

1. Mở đúng tệp Word trong bản clone của kho. Trước khi sửa, cập nhật nhánh và xác nhận không có người đang chỉnh cùng bản mà chưa thống nhất.
2. Sửa, lưu và đóng Word nếu tệp bị khóa hoặc Desktop chưa nhận thay đổi. Các tệp tạm của Word thường bắt đầu bằng `~$`; chúng không phải tài liệu cần phát hành.
3. Đọc phần nội dung đã sửa trong Word. Với tệp nhị phân, đừng trông chờ Git hiển thị diff văn bản đầy đủ.
4. Commit tệp `.docx` với thông điệp mô tả mục nội dung đã đổi, rồi push và mở PR theo quy trình.

Kết quả: nhóm có bản tài liệu mới trong lịch sử kho. Tránh tạo liên tục các tên `ban-cuoi`, `ban-cuoi-moi`, `ban-cuoi-that`; giữ một tên tệp ổn định và dùng commit để lưu phiên bản.

Tài liệu chứa ảnh lớn có thể làm kho tăng dung lượng nhanh vì lịch sử vẫn giữ các phiên bản cũ. Với nhiều tệp lớn, xem giới hạn hiện hành của GitHub, Git LFS và nhu cầu thực tế trước khi chọn cách lưu. LFS có cơ chế dung lượng và băng thông riêng; không xem nó là nơi lưu vô hạn miễn phí.

12.3. .gitignore: bỏ qua tệp không nên theo dõi

Bạn sẽ làm gì? Tạo quy tắc để các tệp sinh tạm hoặc cấu hình riêng chưa được theo dõi không thường xuyên xuất hiện trong danh sách cần commit.

Vì sao cần? Tệp tạm, thư viện cài trên máy và thông tin riêng làm kho rối hoặc gây lộ dữ liệu. .gitignore là tệp văn bản được lưu trong kho để cả nhóm dùng quy tắc chung. Nó không mã hóa dữ liệu và không phải công cụ thu hồi bí mật.

Ví dụ nhỏ, chỉ thêm những quy tắc phù hợp dự án:

```
# Tep tam cua Word
~$*.docx

# Cau hinh bi mat tren may
.env
.env.*
!.env.example

# Tep he dieu hanh
.DS_Store
Thumbs.db
```

Nếu sau này dự án dùng Node.js hoặc Python, tham khảo mẫu phù hợp để bỏ qua node_modules/ hoặc .venv/. Không bỏ qua mã nguồn, tệp tài liệu cần chia sẻ hay lockfile chỉ vì chúng khó hiểu; lockfile thường giúp cài đúng phiên bản phụ thuộc.

Kiểm tra: tạo hoặc xem một tệp tạm không có dữ liệu nhạy cảm và chạy git status. Muốn biết quy tắc nào áp dụng cho một đường dẫn, có thể dùng git check-ignore -v với đúng tên tệp. Không cần tạo một tệp bí mật thật chỉ để thử ignore.

12.4. Tệp đã được theo dõi không tự biến mất vì thêm ignore

Nếu tệp đã commit, thêm nó vào .gitignore không ngừng theo dõi tệp đó. Với một tệp cấu hình cá nhân **không chứa bí mật** mà nhóm thống nhất cần giữ trên máy nhưng bỏ khỏi kho, thực hiện có kiểm tra:

1. Sao lưu và xác nhận tên tệp chính xác.
2. Thêm đúng đường dẫn vào .gitignore.
3. Dùng tên tệp hẹp, ví dụ giả định local-settings.txt là tệp cần bỏ theo dõi:

```
git rm --cached -- local-settings.txt
git status
git diff --cached
```

4. Kiểm tra .gitignore, đưa nó vào commit, commit rồi push qua quy trình dự án.

Điều sẽ xảy ra: --cached bỏ tệp khỏi phần theo dõi của Git và giữ tệp trên máy hiện tại. Commit tiếp theo ghi nhận việc xóa tệp khỏi phiên bản của kho; người khác cập nhật kho có thể bị ảnh hưởng. Cần trao đổi nếu họ cần giữ bản cục bộ. Lịch sử cũ vẫn có tệp; thao tác này không xóa dữ liệu khỏi mọi commit trước đó.

Nếu tệp chứa mật khẩu, token hoặc khóa đã được đưa lên GitHub, hãy xem đó là thông tin đã lộ: thu hồi hoặc đổi thông tin trước, rồi làm theo quy trình xử lý lịch sử của GitHub và nhóm. Chỉ dùng .gitignore hoặc git rm --cached không đủ.

12.5. Chia sẻ cách cấu hình mà không chia sẻ bí mật

Nếu ứng dụng cần biến môi trường, tạo .env.example chỉ chứa tên biến và giá trị giả hoặc chỗ trống, chẳng hạn:

```
APP_MODE=development
API_KEY=THAY_BANG_GIA_TRI_CUA_BAN
```

README giải thích lấy giá trị từ đâu và đặt trong môi trường nào. Mỗi người tự tạo `.env` trên máy theo hướng dẫn; giá trị thật không đưa vào kho. Với môi trường đám mây, ưu tiên phần quản lý secrets và variables của nền tảng, không chèn bí mật vào script cài đặt.

Kết quả cần đạt sau chương này: kho có tài liệu dễ tìm, tệp cần chia sẻ được theo dõi, tệp tạm được bỏ qua và không chứa bí mật. Trước mỗi commit, vẫn đọc `git status` và diff; một quy tắc ignore không thay thế việc kiểm tra của bạn.

Chương 13. Quản lý công việc bằng Issues, Projects và milestone

Một kho mã cần cả mã nguồn và việc cần làm

Issue là phiếu công việc có tiêu đề, mô tả, trao đổi và trạng thái mở hoặc đã đóng. Dùng issue để theo dõi lỗi, đề xuất hoặc việc cần làm. Issue không tự sửa mã.

Ví dụ: “Viết hướng dẫn cài đặt” là một việc; “Nút Lưu không hoạt động” là báo lỗi. Mỗi issue nên có kết quả có thể kiểm tra. Không đưa mật khẩu, token hoặc dữ liệu riêng vào issue.

Tạo một issue dễ hiểu

Mở kho mã, chọn **Issues** → **New issue**. Nếu dự án có mẫu, chọn mẫu phù hợp. Nếu không thấy tab Issues, kiểm tra bạn có mở đúng kho và tính năng này có được bật không. Với bài luyện tập, tạo issue “Bổ sung hướng dẫn sử dụng DQY” rồi điền:

```
Mục tiêu: Người mới mở kho biết dự án dùng để làm gì.
Việc cần làm:
- [ ] Viết mục tiêu của dự án.
- [ ] Nêu cách đọc tài liệu.
- [ ] Kiểm tra đường dẫn trong README.
Hoàn thành khi: README có đủ ba nội dung và người khác đọc hiểu.
```

Bấm nút tạo và ghi lại số issue. Khi viết PR, dùng `Closes #1` với số thực tế để liên kết và đóng issue khi PR được hợp nhất vào nhánh mặc định theo quy tắc GitHub. Chỉ đóng khi kết quả đã đạt.

Báo lỗi khác đề xuất tính năng

Một báo lỗi tốt giúp người khác tái hiện điều bạn gặp. Một yêu cầu tính năng tốt giải thích nhu cầu, tránh chỉ ra lệnh “thêm cái này”.

Loại	Nội dung nên có	Ví dụ
Báo lỗi	Các bước, kết quả mong đợi, kết quả thực tế, môi trường	“Mở trang A, bấm Lưu; mong có thông báo; thực tế không phản hồi.”
Đề xuất	Người dùng nào cần, vấn đề hiện tại, kết quả mong muốn	“Tôi muốn tìm tài liệu theo tên để mở nhanh khi danh sách dài.”
Công việc	Phạm vi, bước thực hiện, tiêu chí hoàn thành	“Thêm mục cài đặt vào README; kiểm tra bằng một lần làm thử.”

Che dữ liệu riêng trong ảnh minh họa. Nếu chưa chắc nguyên nhân, chỉ mô tả điều quan sát được.

Labels, người phụ trách và milestone

Label là nhãn để lọc, ví dụ `bug`, `documentation`, `enhancement`. Tên nhãn tùy kho; không phải mọi kho đều có cùng danh sách. **Assignee** là người chịu trách nhiệm chính. **Milestone** nhóm việc theo một mục tiêu hoặc phiên bản, chẳng hạn “Tài liệu cơ bản v1”. Quyền tạo hoặc chỉnh sửa các mục này phụ thuộc vai trò của bạn.

Bắt đầu bằng vài nhãn. Milestone phù hợp khi nhiều issue cùng phục vụ một mốc, không bắt buộc cho thay đổi nhỏ.

Projects là bảng nhìn toàn cảnh

GitHub Projects hiển thị bảng, trạng thái hoặc lộ trình. Bạn có thể nhóm issue và PR thành “Chưa làm”, “Đang làm”, “Xong”. Project có thể chứa việc của nhiều kho, tùy quyền.

Thử thêm một issue vào Project. Đổi cột không nhất thiết tự đóng issue; xem trạng thái issue và quy tắc tự động hóa đã cấu hình.

Bài thực hành: tạo một issue tài liệu, đặt tiêu chí hoàn thành, làm thay đổi trên nhánh riêng, mở PR và liên kết đúng issue. Bạn đạt mục tiêu khi giải thích được “việc cần làm ở đâu” và “mã thay đổi để làm việc đó ở đâu”.

Chương 14. Actions, Pages và Releases: hiểu trước khi tự động hóa

CI và CD giải quyết việc gì?

CI, tích hợp liên tục, tự chạy kiểm tra khi có thay đổi: kiểm thử, định dạng hoặc xây dựng chương trình. Kết quả xanh chỉ cho biết kiểm tra đã cấu hình đều đạt; không chứng minh chương trình không còn lỗi.

CD liên quan đến chuẩn bị phát hành hoặc triển khai. Actions không bắt buộc tự triển khai; dự án mới có thể chỉ kiểm tra rồi phát hành thủ công.

Đọc một workflow

Workflow GitHub Actions thường là tệp YAML trong `.github/workflows/`. **Event** là sự kiện kích hoạt, **job** là nhóm việc, **step** là bước trong job, **runner** là máy chạy job. Một step có thể chạy lệnh hoặc dùng một action được đóng gói sẵn.

Ví dụ dưới đây kiểm tra có tệp README không rỗng. Đây là minh họa để đọc cấu trúc, không phải cấu hình đã được cài hay chạy cho DQY, cũng không triển khai ứng dụng.

```yaml
name: Check README
on: [push, pull_request]
permissions:
  contents: read
jobs:
  documentation:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - name: Check README exists and has content
        run: test -s README.md
```

`on` nói khi nào chạy; `permissions` giới hạn quyền token; `checkout` lấy mã; `run` thực hiện lệnh. Kiểm tra này không đánh giá chất lượng tài liệu. Dự án thật cần phiên bản action được hỗ trợ; có thể ghim action bằng mã commit đầy đủ để kiểm soát phiên bản.

Sau khi hiểu nhánh và PR, thử trên nhánh thực hành. Vào **Actions**, mở lần chạy, job rồi bước để đọc log. Tìm bước đỏ đầu tiên, sửa nguyên nhân, commit rồi kiểm tra lần chạy mới.

Quyền, secrets và mã đến từ người khác

Workflow có thể dùng token GitHub tự cấp và các **secret** bạn cấu hình trong cài đặt kho, môi trường hoặc tổ chức. Chỉ cấp quyền cần thiết. Không đặt token trong YAML, không in token vào log và không gửi token qua issue. Việc che secret trong log có giới hạn; nó không thay thế cho việc bảo vệ dữ liệu.

PR từ fork có quy tắc quyền và secret khác với PR từ nhánh trong cùng kho. Theo cấu hình thông thường, secret không được đưa cho workflow của PR từ fork; một số sự kiện hoặc thiết lập có hành vi khác. Đừng đổi sang sự kiện quyền cao chỉ để “cho chạy được”, đặc biệt khi workflow tải và chạy mã do người ngoài gửi. Hãy học kỹ mô hình tin cậy trước khi dùng `pull_request_target`.

Runner GitHub thường là máy tạm. Cấu hình cache/artifact nếu cần giữ dữ liệu; không lưu secret thành artifact. Runner tự quản lý cần được bảo vệ. Chi phí, phút chạy và lưu trữ tùy gói, kho và thiết lập; kiểm tra **Billing and licensing**.

Pages dành cho website tĩnh

GitHub Pages phù hợp với HTML, CSS, JavaScript phía trình duyệt và tài liệu được xây dựng thành tệp tĩnh. Nó không cung cấp máy chủ ứng dụng để chạy Python, PHP, Node.js phía máy chủ hoặc cơ

sở dữ liệu của bạn. Website tĩnh vẫn có thể gọi một API ở nơi khác, nhưng dịch vụ API cần được thiết lập riêng.

Thử chuẩn bị index.html, rồi xem **Settings → Pages** để chọn cách xuất bản phù hợp. Chỉ bật khi hiểu nội dung nào công khai. Kho riêng không tự động có nghĩa website riêng; xuất bản từ kho riêng và kiểm soát truy cập tùy gói, tổ chức.

Không đưa khóa bí mật vào JavaScript gửi đến trình duyệt. Mã đó được người truy cập tải về. Không hứa rằng Pages luôn miễn phí cho mọi loại kho; hãy xem điều kiện và giới hạn hiện tại của tài khoản.

Bài thực hành tùy chọn: xuất bản một trang Pages

Chỉ làm bài này trên **kho luyện tập bạn sở hữu và có quyền chỉnh cài đặt**. Nội dung có thể được đưa lên Internet; không dùng tài liệu riêng, mật khẩu hoặc dữ liệu khách hàng. Nếu kho của nhóm có chính sách khác, làm theo chính sách đó. Đây là bài tự thực hiện, không phải thông báo rằng DQY đã được xuất bản.

1. Mở kho luyện tập, chọn nhánh mặc định. Trong ô chọn nhánh, nhập docs/website và chọn tạo nhánh từ nhánh nền đúng. Nếu nhánh đã tồn tại, chọn tên khác.
2. Khi đang ở nhánh mới, chọn **Add file → Create new file**, đặt tên index.html ở thư mục gốc. Dán mười dòng dưới đây, rồi **Commit changes** vào nhánh thực hành.

```
<!doctype html>
<html lang="vi">
<head>
 <meta charset="utf-8">
 <title>Trang luyện tập GitHub</title>
</head>
<body>
 <h1>Tôi đang học GitHub</h1>
</body>
</html>
```

3. Mở **Pull requests → New pull request**. Base là nhánh đích, ví dụ main; compare là docs/website. Xem diff, tạo PR, kiểm tra nội dung rồi merge theo quyền và quy tắc kho. Xác nhận index.html đã nằm trên nhánh đích.
4. Vào **Settings → Pages**. Nếu có lựa chọn, chọn **Source: Deploy from a branch**, chọn nhánh chứa tệp, ví dụ main, và thư mục **/(root)**, rồi **Save**. Dùng tên nhánh thực tế; nếu nhánh mặc định là tên khác, không chọn main chỉ vì ví dụ ghi vậy.
5. Xem trạng thái tại Pages và mở tab **Actions** để theo dõi lần xây dựng/triển khai liên quan. Đợi hoàn tất; nếu đỏ, mở bước lỗi và đọc nguyên nhân. Không bấm Save nhiều lần để xử lý lỗi chưa rõ.
6. Quay lại **Settings → Pages**, dùng liên kết website **do giao diện thực tế cung cấp**. Mở liên kết và kiểm tra xuất hiện tiêu đề “Tôi đang học GitHub”. Không tự đoán địa chỉ: đường dẫn còn tùy tên kho, tài khoản và tên miền cấu hình.

Nếu lựa chọn nguồn không có, bị khóa hoặc báo hạn chế, dừng phần xuất bản và xem quyền, chính sách tổ chức, loại kho và gói tài khoản. Không đổi kho private thành public chỉ để vượt hạn chế. Bài đạt khi trang mở được đúng nội dung và bạn xác nhận dữ liệu được phép công khai. Đây chỉ là HTML tĩnh; nó không tạo backend hay cơ sở dữ liệu.

Tag và Release khác nhánh

Nhánh di chuyển khi bạn thêm commit; **tag** đánh dấu một commit cụ thể, thường cho phiên bản như v1.0.0. **Release** là trang phát hành trên GitHub dựa trên tag, có ghi chú và có thể kèm tệp tải xuống. Nó không tự đưa website lên mạng và cũng không tự cài phần mềm cho người dùng.

Khi có mốc ổn định, kiểm tra mã, chọn commit, tạo tag và ghi thay đổi, cách dùng, giới hạn. Ký tag hỗ trợ xác minh nguồn gốc, có thể học sau. Đánh dấu **pre-release** cho bản thử. Đừng âm thầm thay tag đã được sử dụng.

Bài thực hành tùy chọn: tạo một Release

Release đưa một mốc đã kiểm tra thành bản phát hành dễ tìm. Chỉ làm khi có quyền phát hành và đã xem phạm vi hiển thị của kho. **Publish release là thao tác xuất bản trực tuyến**; trong kho public, nội dung và tệp kèm có thể được người khác tải. Hướng dẫn này không tự xuất bản thay bạn.

1. Kiểm tra nhánh đích đã có nội dung ổn định, PR cần thiết đã merge và kiểm tra đã đạt. Ghi lại commit định phát hành; nếu kho thay đổi liên tục, dùng đúng SHA thay vì chỉ nhớ tên nhánh.
2. Từ trang kho, mở **Releases**, chọn **Draft a new release**. Vị trí nút có thể thay đổi; tìm trang quản lý bản phát hành của kho.
3. Trong phần chọn tag, nhập `v0.1.0` nếu tag này chưa tồn tại và chọn tạo tag mới khi phát hành. Chọn **Target** đúng nhánh, chẳng hạn `main`, hoặc commit định phát hành. Kiểm tra lại target trước khi tiếp tục. Tag hiện có phải trỏ tới đúng commit; không thay tag cũ để tái sử dụng số phiên bản.
4. Nhập tiêu đề, ví dụ “DQY v0.1.0 — tài liệu ban đầu”, và ghi chú: thay đổi gì, cách sử dụng, giới hạn đã biết. Tên DQY chỉ dùng nếu bạn thực sự đang phát hành kho DQY; kho luyện tập có tên riêng thì dùng tên đó.
5. Có thể đính kèm `.docx` bằng vùng tải tệp của Release. Mở bản định đính kèm trước, kiểm tra thông tin riêng, bình luận, chỉnh sửa theo dõi và thuộc tính tài liệu. Đối chiếu giới hạn hiện hành của tệp đính kèm bản phát hành; tệp không đạt thì chưa tải lên.
6. Nếu đây là bản thử, bật **Set as a pre-release**. Bạn có thể lưu draft để kiểm tra thêm. Chỉ bấm **Publish release** khi đồng ý xuất bản đúng nội dung và đúng phạm vi.
7. Mở trang Release vừa tạo, kiểm tra tag và ghi chú, tải tệp đính kèm về rồi mở lại. Bài đạt khi đúng phiên bản, đúng commit và tài liệu đọc được. Nếu phát hiện lỗi sau phát hành, xử lý minh bạch theo quy trình dự án; không âm thầm chuyển tag sang commit khác.

Chương 15. Máy cá nhân, môi trường đám mây và tệp Word

Chọn nơi làm việc

Bạn có thể làm việc trên website, máy cá nhân hoặc môi trường đám mây. Chọn cách đủ cho công việc, không cần cài mọi công cụ.

Cách	Phù hợp	Điều cần nhớ
GitHub trên trình duyệt	Sửa README, xem mã, issue, PR	Không tự cung cấp đầy đủ môi trường chạy ứng dụng
Máy cá nhân	Làm việc thường xuyên, chạy và kiểm thử	Tự cài Git, công cụ dự án và bảo vệ máy
GitHub Codespaces	Môi trường phát triển do GitHub cung cấp	Khả năng dùng, cấu hình, hạn mức và chi phí tùy tài khoản
Codex trong môi trường đám mây	Giao việc hỗ trợ đọc, sửa, kiểm tra mã	Xác minh kết quả, quyền truy cập và cách lưu thay đổi của phiên làm việc

Codex và Codespaces có quyền sử dụng riêng. Máy đám mây vẫn cần đúng công cụ và thư viện để chạy dự án.

Điều gì được giữ sau khi dừng phiên?

Một commit chỉ nằm ở máy làm việc cho đến khi được push thành công. Tệp chưa commit hoặc commit chưa push có thể bị mất nếu máy bị xóa. Dừng môi trường không đồng nghĩa với lưu mọi thay đổi lên GitHub. Trước khi kết thúc, xem `git status`, xem commit cần đẩy và xác nhận trên website.

Ảnh môi trường hoặc đĩa được giữ lại không nhất thiết khôi phục tiến trình, terminal, biến tạm và đăng nhập. Lưu cách cài đặt, khởi động; đừng dựa vào việc “hôm qua server vẫn chạy”.

Trong Codex cloud, kiểm tra phạm vi lưu trữ của phiên và dùng checkout được cấp. Tác vụ đã có môi trường riêng; không cần tạo Git worktree nếu không có nhu cầu cụ thể. Hướng dẫn thiết lập không được chứa bí mật.

GitHub lưu Word được, nhưng không phải trình soạn Word trực tiếp

`.docx` là tệp nhị phân. Bạn thường soạn bằng Microsoft Word hoặc phần mềm tương thích, lưu lại rồi tải lên GitHub hoặc commit bằng Git. GitHub có lịch sử tệp, nhưng không hiển thị so sánh từng câu và hợp nhất thay đổi Word thuận tiện như tệp văn bản.

Với người mới, giao một người chỉnh một tệp Word tại một thời điểm. Người đó tải bản mới nhất, sửa, lưu, tải lên đúng đường dẫn và viết commit mô tả. Người kế tiếp lấy phiên bản sau khi việc trước hoàn tất. Nếu cần nhiều người cùng gõ và bình luận trực tiếp trong tài liệu, dùng chức năng cộng tác phù hợp của Microsoft 365 hoặc công cụ tài liệu tương ứng; GitHub không thay thế cách làm đó.

Đặt tên ổn định như `docs/Huong-dan-GitHub.docx`. Lịch sử commit giữ các phiên bản; không cần tạo hàng loạt tên `final`, `final2`, `final-that`. Nếu muốn một bản phát hành cố định, có thể đính kèm tài liệu vào Release. Tải và mở kiểm tra tài liệu sau khi lưu để chắc bạn đã đưa đúng bản lên.

Tệp lớn và Git LFS

Git thường không phù hợp để lưu hàng loạt video, dữ liệu nặng hoặc nhiều lần sửa tệp nhị phân lớn. Xóa một tệp ở commit mới không xóa tệp đó khỏi lịch sử cũ; kho vẫn có thể nặng. Đừng dùng GitHub như nơi sao lưu toàn bộ ổ đĩa.

Với Git thông thường, GitHub cảnh báo khi tệp vượt 50 MiB và chặn tệp lớn hơn 100 MiB. Tải bằng trình duyệt có giới hạn riêng, thường tối đa 25 MiB cho mỗi tệp. Kiểm tra tài liệu “About large files on GitHub” khi thao tác vì sản phẩm và gói có thể có giới hạn bổ sung.

Git LFS lưu một tệp con trỏ nhỏ trong Git và quản lý nội dung lớn ở nơi lưu trữ riêng. Nó có hạn mức, chi phí và kích thước tối đa theo gói. Cộng tác viên cần cài hoặc cấu hình phù hợp để lấy nội dung thật. LFS không làm mọi tệp trở nên miễn phí, không tự sửa toàn bộ lịch sử có sẵn và không phải nơi chứa bí mật.

Word nhỏ có thể dùng Git thường; video/dữ liệu lớn nên cân nhắc lưu trữ chuyên dụng và ghi liên kết trước khi đưa vào lịch sử.

Chương 16. Xử lý lỗi theo dấu hiệu, không đoán mò

Quy trình bốn bước

Khi lỗi, ghi thông báo, lệnh và thư mục hiện tại. Kiểm tra `git status`, `git remote -v`, `git branch --show-current`. Che thông tin nhạy cảm trước khi chia sẻ; không gửi toàn bộ môi trường hoặc tệp xác thực.

Sửa một nguyên nhân có bằng chứng rồi thử lại. Trước thao tác khó, sao lưu tệp chưa commit ở nơi riêng; đừng đưa bản sao đó vào kho.

Bảng lỗi thường gặp

Dấu hiệu	Nguyên nhân có thể	Cách kiểm tra và sửa	Khi nào đạt
`not a git repository`	Terminal ở sai thư mục	Xem thư mục hiện tại, chuyển vào thư mục kho đã clone; tránh chạy `git init` chỉ để hết lỗi	`git status` hiển thị kho đúng
`nothing to commit`	Chưa lưu tệp, sai kho, tệp bị ignore hoặc không có khác biệt	Lưu trong trình soạn, xem `git status`, `git diff`; kiểm tra `.gitignore` nếu cần	Nhìn thấy thay đổi dự định, hoặc xác nhận thật sự không có thay đổi
404 hoặc “Repository not found”	URL sai, kho đã đổi tên, kho riêng hoặc thiếu quyền	Mở URL bằng tài khoản đúng, kiểm tra quyền và remote; kho riêng có thể trả 404	Đọc được kho cần dùng bằng cơ chế đã xác thực
403 hoặc “permission denied”	Không có quyền ghi, token thiếu quyền, chính sách tổ chức hoặc SSH sai	Xác định đang dùng HTTPS hay SSH; kiểm tra tài khoản, quyền, yêu cầu SSO nếu có	Push tới kho được phép hoặc mở PR từ fork
Push bị `non-fast-forward`	Nhánh remote có commit máy bạn chưa có	`git fetch`, xem lịch sử; theo quy trình dự án để pull/merge và giải quyết khác biệt	Push được, vẫn giữ các thay đổi cần thiết của người khác
Xung đột merge	Hai phía sửa phần chồng nhau	Xem `git status`, mở tệp, chọn nội dung đúng, xóa dấu xung đột, kiểm tra rồi commit	Nội dung đúng, không còn tệp unmerged
Cảnh báo LF/CRLF	Khác kiểu xuống dòng giữa hệ điều hành hoặc cấu hình	Xem chính sách `.gitattributes` của kho; tránh đổi cấu hình toàn cục vội	Không xuất hiện thay đổi cả tệp chỉ do xuống dòng
`detached HEAD`	Đang đứng trực tiếp ở commit/tag	Nếu muốn giữ công việc, tạo nhánh bằng `git switch -c ten-nhanh` trước khi rời	Commit mới nằm trên nhánh có tên
Push bị chặn vì tệp lớn	Tệp vượt giới hạn hoặc vẫn nằm trong commit trước	Đọc thông báo, tìm commit chứa tệp; chọn LFS/lưu trữ phù hợp hoặc nhờ xử lý lịch sử	Push thành công và kho không chứa tệp lớn ngoài dự kiến
Workflow đỏ	Lệnh, phụ thuộc, quyền hoặc cấu hình không đúng	Mở bước lỗi đầu tiên trong log, so với hướng dẫn dự án	Lần chạy mới đạt kiểm tra cần thiết

HTTPS GitHub không dùng mật khẩu tài khoản để xác thực thao tác Git. Tùy môi trường, dùng trình quản lý thông tin xác thực, GitHub CLI, token được cấp đúng quyền hoặc cơ chế Git được nền tảng cung cấp. Không xin thêm token chỉ vì chưa thấy biến `GH_TOKEN`: môi trường cloud có thể đã cấp xác thực theo cách khác.

Khi có xung đột

Các dấu `<<<<<<<`, `=======`, `>>>>>>>` đánh dấu xung đột. Đọc mục tiêu hai phía, sửa thành nội dung đúng, xóa các dấu, kiểm tra rồi `git add`. Đừng giữ cả hai đoạn nếu kết quả sẽ sai.

`git merge --abort` có thể quay về trước merge. Rebase cần lệnh riêng. Kiểm tra và sao lưu thay đổi chưa commit vì quay lui có thể có giới hạn. Chưa hiểu trạng thái thì nhờ hỗ trợ với thông báo đã che dữ liệu riêng.

Nếu lỡ đưa bí mật lên GitHub

Thu hồi hoặc đổi bí mật ngay tại nơi cấp nó. Ví dụ token bị lộ phải được revoke; mật khẩu bị lộ phải được đổi theo dịch vụ liên quan. Xóa tệp hoặc làm kho riêng không khiến bí mật cũ an toàn trở lại. Sau đó kiểm tra nơi bí mật có thể xuất hiện: lịch sử, PR, log, artifact và bản sao.

Dọn lịch sử là bước riêng cần phối hợp với người quản lý và cộng tác viên. Nó có thể đổi mã commit, ảnh hưởng nhánh và bản clone. Không tự chạy force push khi chưa hiểu hậu quả. Dùng hướng dẫn chính thức về xóa dữ liệu nhạy cảm, và liên hệ hỗ trợ GitHub nếu cần xử lý các phần còn lưu trên nền tảng. Không gửi chính bí mật đó trong câu hỏi nhờ giúp.

Những lệnh để học sau

`commit --amend`, rebase, reset và force push có ích trong tình huống phù hợp nhưng có thể thay lịch sử hoặc mất thay đổi. Với người mới, ưu tiên commit bổ sung, merge theo quy trình và PR. `--force-with-lease` vẫn là thao tác ghi lại lịch sử; tên của nó không biến mọi lần dùng thành an toàn.

Chương 17. Lộ trình 21 ngày và bài tổng hợp DQY

Học bằng kết quả nhỏ

Dành 20–45 phút/ngày; chưa xong thì tiếp tục hôm sau. Mục tiêu là tự làm, giải thích lý do và kiểm tra kết quả. Ghi lại việc đã làm, lỗi gặp và bước kế tiếp.

Ưu tiên web, Git và cộng tác. Actions, Pages, Releases, LFS có thể học sau. Luyện trong kho riêng hoặc nhánh rõ tên.

Ngày	Việc thực hành	Bằng chứng hoàn thành
1	Bảo vệ tài khoản, kiểm tra email và xác thực hai bước	Biết cách đăng nhập và nơi lưu mã khôi phục an toàn
2	Xem repo, nhánh, README và lịch sử	Giải thích được bốn mục này bằng lời của mình
3	Sửa README trên web và commit	Xem được thay đổi trong lịch sử
4	Tạo thư mục tài liệu và tải một tệp nhỏ	Tệp ở đúng đường dẫn, mở/tải được
5	Cài Git, cấu hình danh tính phù hợp	Kiểm tra được phiên bản và thông tin commit
6	Clone kho về máy hoặc môi trường được cấp	`git status` nhận đúng kho
7	Sửa, xem diff, add và commit	Commit chỉ chứa thay đổi dự định
8	Push rồi kiểm tra trên GitHub	Website có đúng commit và nhánh
9	Fetch/pull khi working tree sạch	Hiểu lấy thông tin khác với hợp nhất
10	Tạo nhánh và đổi nhánh	Có nhánh thực hành, không mất tệp
11	Mở PR với mô tả và cách kiểm tra	PR có phạm vi nhỏ, đúng base/compare
12	Đọc diff và phản hồi review	Sửa bằng commit mới, trao đổi rõ lý do
13	Merge PR, cập nhật máy làm việc	Nhánh chính có thay đổi đã duyệt
14	Thử xung đột trong kho luyện tập	Giải quyết được và nội dung cuối đúng
15	Tạo issue tài liệu và tiêu chí đạt	Người khác biết khi nào việc hoàn tất
16	Thử labels hoặc Project nếu cần	Theo dõi được một việc từ mở đến xong
17	Đọc một workflow đơn giản	Chỉ ra được trigger, job, step, runner
18	Đọc log Actions; thử Pages nếu cần	Biết tìm lỗi; hiểu phạm vi xuất bản
19	Đọc tag/Release và giới hạn tệp	Phân biệt bản làm việc với bản phát hành
20	Làm bài DQY bên dưới	PR tài liệu đã sẵn sàng review
21	Tự rà soát và ghi quy trình cá nhân	Lặp lại được workflow không cần chép mù

Bài tổng hợp: tài liệu sử dụng DQY

Mục tiêu là thay đổi tài liệu có dấu vết đầy đủ. Nếu chưa có ứng dụng, không bịa lệnh cài đặt hoặc tuyên bố đã chạy.

1. Mở `nix6242/DQY`, kiểm tra quyền, nhánh mặc định và nội dung hiện có.
2. Tạo issue “Hoàn thiện hướng dẫn người mới”. Nêu ba tiêu chí: mô tả dự án, cách đọc tài liệu, cách đóng góp một thay đổi.
3. Clone nếu chưa có bản sao; dùng checkout được cấp nếu đã có. Kiểm tra trạng thái, cập nhật nhánh nền theo quy trình.
4. Tạo nhánh `docs/huong-dan-nguoi-moi` từ nền phù hợp; chọn tên khác nếu đã tồn tại.

5. Sửa README: mục tiêu, đường dẫn tài liệu. Nếu đưa Word vào `docs/Huong-dan-GitHub.docx`, kiểm tra kích thước và nội dung riêng.
6. Xem `git status`, `git diff` và mở lại Word. Add đúng tệp, commit với thông điệp rõ; tránh stage tệp không liên quan.
7. Push nhánh thực hành. Mở PR với base là nhánh đích của dự án. Mô tả kết quả, cách kiểm tra, liên kết đúng issue bằng số thực tế.
8. Xem **Files changed**, đường dẫn và tải lại Word. Xử lý kiểm tra bắt buộc nếu có; ghi kiểm tra thủ công đã làm.
9. Review trước khi merge. Không có quyền thì người quản lý xử lý; phần đóng góp của bạn vẫn được ghi nhận trong PR.
10. Sau merge, cập nhật local/cloud. Chỉ xóa nhánh khi thay đổi đã được giữ và không còn công việc cần lưu.

Bảng tự nghiệm thu

- Tôi biết kho nào, nhánh nào và tài khoản nào mình đang dùng.
- Tôi giải thích được clone, commit, push, pull và PR khác nhau thế nào.
- Commit không có token, mật khẩu hoặc tệp không liên quan.
- Issue có tiêu chí; PR có mô tả và cách kiểm tra thật.
- Tài liệu Word mở được, README có đường dẫn đúng.
- Tôi xác nhận kết quả trên GitHub, không chỉ nhìn thông báo ở terminal.
- Tôi biết phần nào chưa làm và không tuyên bố kiểm thử ứng dụng khi chưa có ứng dụng.

Chương 18. Tra nhanh: lệnh, thuật ngữ và câu hỏi thực tế

Bộ lệnh thường dùng

Chạy trong đúng kho, thay tên mẫu và đọc kết quả từng bước. Add, commit, push là thao tác thay đổi trạng thái.

Mục đích	Lệnh	Lưu ý
Xem trạng thái	`git status`	Bước đầu khi chưa chắc điều gì đang xảy ra
Xem sửa chưa stage	`git diff`	Chủ yếu hữu ích cho tệp văn bản
Xem nội dung đã stage	`git diff --staged`	Kiểm tra trước commit
Xem nhánh hiện tại	`git branch --show-current`	Kết quả trống có thể là detached HEAD
Xem nơi kết nối	`git remote -v`	Che URL nếu có thông tin xác thực
Tạo và chuyển nhánh	`git switch -c docs/huong-dan`	Bắt đầu từ nhánh nền đúng
Chuyển nhánh đã có	`git switch main`	Dùng tên nhánh thật; kiểm tra tệp chưa commit
Stage một tệp	`git add README.md`	Không mặc định add tất cả
Lưu commit	`git commit -m "Bo sung huong dan"`	Commit nằm local cho đến khi push
Đẩy nhánh lần đầu	`git push -u origin docs/huong-dan`	Cần xác thực và quyền ghi phù hợp
Lấy thông tin mới	`git fetch origin`	Chưa hợp nhất vào nhánh đang làm
Cập nhật đơn giản	`git pull --ff-only`	Chỉ khi phù hợp quy trình; dừng nếu lịch sử đã phân kỳ
Xem lịch sử ngắn	`git log --oneline -5`	Đọc năm commit gần nhất

`git pull --ff-only` dừng nếu không thể cập nhật theo đường thẳng. Xem lịch sử và quy trình dự án, không force push để hết lỗi.

Từ điển ngắn

Thuật ngữ	Cách hiểu
Working tree	Các tệp đang có trong thư mục làm việc
Staging area	Những thay đổi bạn đã chọn cho commit kế tiếp
Remote / origin	Kết nối đến kho khác; `origin` là tên thường dùng
Fork	Bản kho trên GitHub thuộc tài khoản hoặc tổ chức khác
Pull request	Đề nghị xem xét và hợp nhất thay đổi giữa nhánh
.gitignore	Quy tắc bỏ qua tệp chưa được Git theo dõi
Artifact	Tệp kết quả được lưu từ một lần chạy workflow

`.gitignore` không tự xóa tệp đã theo dõi hoặc lịch sử. Bỏ bí mật hay giảm dung lượng cần xử lý riêng.

Những câu hỏi thường gặp

Lưu Word rồi, GitHub có bản mới chưa? Chưa chắc. Save, commit và push khác nhau. Upload trên web cũng phải hoàn tất commit. Tải lại để kiểm tra.

Commit có giống Save không? Commit lưu thay đổi vào lịch sử; không thay Save trong trình soạn và không tự gửi lên GitHub.

Một mình có cần PR không? Không luôn bắt buộc, nhưng hữu ích để tập review. Khi làm nhóm, tuân thủ quy trình và quy tắc nhánh.

Private có bảo đảm bí mật an toàn không? Vẫn cần quản lý quyền, bản clone, log và token. Khóa đã lộ phải thu hồi, không chỉ đổi kho sang private.

Chưa biết lập trình có học được không? Có. Luyện bằng README và Word để học lịch sử, nhánh, PR, issue; học chạy và kiểm thử khi có mã.

Dùng trợ lý viết mã cần kiểm tra gì? Đọc phạm vi, lý do, diff, kết quả thật. Nếu không hiểu lệnh, yêu cầu giải thích trước khi chạy.

Giao diện khác tài liệu thì sao? Nút, gói và quyền có thể đổi. Tìm theo mục đích, đối chiếu GitHub Docs; phần thiếu quyền cần người quản lý xử lý.

Nguồn tham khảo và cách tra cứu tiếp

Tài liệu chính thức đã dùng để đối chiếu

Các phần giải thích được biên soạn bằng tiếng Việt cho việc học thực hành. Nguồn đối chiếu là GitHub Docs và tài liệu của action chính thức; đây không phải bản dịch chính thức của GitHub. Đã đọc các bài dưới đây trong kho nguồn công khai `github/docs` ngày 11/10/2026, ở commit `be38ec5d78e24172587e61b3c6ff40ace1865c71`. Tên nút và khả năng sử dụng vẫn phụ thuộc quyền, cấu hình và gói tài khoản của bạn.

Bạn có thể Ctrl + bấm liên kết trong Word để mở trình duyệt, tùy cài đặt của ứng dụng. Nếu GitHub đổi đường dẫn, tìm tên bài tại GitHub Docs.

- Hello World.
- GitHub flow.
- About pull requests.
- Resolving a merge conflict using the command line.
- About large files on GitHub.
- Caching your GitHub credentials in Git.
- About two-factor authentication.
- Configuring two-factor authentication recovery methods.
- Ignoring files.
- Removing sensitive data from a repository.
- About GitHub Desktop.
- Supported operating systems for GitHub Desktop.
- Understanding GitHub Actions.
- Managing your personal access tokens.
- About Git Large File Storage.
- About releases.
- Configuring a publishing source for your GitHub Pages site.
- What is GitHub Pages?.
- What are GitHub Codespaces?.
- Creating a new repository.
- Adding a file to a repository.
- About Projects.
- About rulesets.
- actions/checkout: tài liệu phiên bản v7 — ví dụ lấy mã xuống runner ở chương 14.

GitHub Docs công bố giấy phép CC BY 4.0 cho nội dung tài liệu; xem giấy phép nội dung. Cẩm nang diễn giải và chọn ví dụ phù hợp người mới, không sao chép toàn bộ các bài nguồn.

Nơi tải công cụ và tra lệnh

- Git: trang tải chính thức — chọn đúng hệ điều hành.
- GitHub Desktop — ứng dụng giao diện cho Windows và macOS.
- GitHub CLI — công cụ `gh`, tùy chọn khi bạn cần.

- Git: tài liệu lệnh — tra chính xác tham số của `status`, `switch`, `merge`, `restore`, `revert` và `stash`.
- Pro Git, sách miễn phí — học tiếp về lịch sử, nhánh và cộng tác.
- GitHub Skills — bài học thực hành do GitHub cung cấp.
- Choose a License — đọc điều kiện giấy phép trước khi chọn.

Các liên kết công cụ và tài nguyên học thêm giúp bạn tra cứu tiếp; cẩm nang không khẳng định đã cài hoặc chạy tất cả công cụ đó trên máy của bạn.

Khi cần nhờ hỗ trợ

Bạn có thể gửi một câu hỏi theo mẫu sau, sau khi bỏ dữ liệu nhạy cảm:

```
Tôi muốn: ...
Tôi dùng: Windows/macOS/Linux hoặc môi trường đám mây ...
Kho và nhánh đang làm: ...
Tôi thực hiện bước/lệnh: ...
Thông báo nhận được: ...
Kết quả tôi mong đợi: ...
Tôi đã kiểm tra: ...
```

Không gửi mật khẩu, token, mã khôi phục, toàn bộ biến môi trường hoặc tệp chứa thông tin xác thực. Nếu là kho private của công ty, chỉ chia sẻ thông tin và mã nguồn theo phạm vi được phép. Một câu hỏi có bước tái hiện và thông báo chính xác giúp người hỗ trợ giải quyết nhanh hơn.